UC Berkeley CS61C 计算机体系结构（Great Ideas in Computer Architecture）学习笔记

课程：UC Berkeley CS 61C *Great Ideas in Computer Architecture (Machine Structures)*，Fall 2026，4 学分。 **讲师**：Dan Garcia（课程主页标注；课堂 Monday/Wednesday/Friday 11:00AM–12:00PM PT，Gateway 1210）。 **课程主页**：https://cs61c.org/ ｜ 本学期主页：https://cs61c.org/fa26/ ｜ 归档：https://inst.eecs.berkeley.edu/~cs61c/archives.html **课程笔记（公开教科书式材料）**：https://notes.cs61c.org/ 。本笔记按 notes 的章节号逐节阅读，覆盖 number-rep、floating-point、c-basics、c-pointers-arrays-strings、c-memory-management、c-generics、rv-*(intro/data-transfer/decision-making/procs/isa-formats-i/ii)*、*call*、*sds-*(intro/combinational-logic/state/blocks)、datapath、pipeline、pipeline-hazards、caches-intro/ii/iii、vm、parallel-*(performance/dlp/tlp)* 共 156 个页面（sitemap 全量抓取，全部 HTTP 200）。 *讲义页：*`https://cs61c.org/fa26/lectures/lecNN/`*（lec01–lec40 全部返回 HTTP 200）。重要事实：这 40 个页面本身只是索引页（约 6 KB），正文只包含讲题、*`Slides`*（Google Slides 链接）、*`Drive`*录像链接与*`Readings`*链接，不含可提取的讲义正文或 PDF。因此本笔记的技术内容以*`notes.cs61c.org`*章节正文为准，讲次信息（讲题、日期、配套 lab/hw/project）以主页 Schedule 表为准。 教材：按本学期 Policies 页* Reading *一节，课程笔记基于 P&H（Patterson & Hennessy,* Computer Organization and Design RISC-V Edition*, 2nd ed., ISBN 0128203315）、K&R（*The C Programming Language*, 2nd ed.）与 WSC（Barroso et al.,* The Datacenter as a Computer*, 2013）三本书。Policies 页同时写明：*this semester, course notes will be the reference for what is in scope——*考试范围以课程笔记为准。 常见说法澄清：坊间常把 CS61C 的教材写成 Patt & Patel《Introduction to Computing Systems: From Bits and Gates to C and Beyond》。该书是另一门课（Berkeley CS61C 早期版本与许多学校的大二计算机组成课）常用的教材；Fall 2026 官方 Policies 页列出的三本参考书里没有 P&P。本文按官方页面写 P&H/K&R/WSC，并在正文需要处标注这一差异。 项目：Project 1*`snek`*（C）、Project 2*`CS61Classify`*（RISC-V 汇编 / 手写数字分类）、Project 3*`CS61CPU`*（Logisim Evolution 搭 RV32I 单周期 CPU）；主页另列一个可选 Project 4*`61kaChow`*。 访问日期：所有 URL 均于 2026-11 访问。文中日期、分数、截止时间均抄自当学期主页 / Policies 页 / 项目页，未做任何推测性补全。* 版权声明：本文档为个人学习笔记。课程材料（课程笔记、讲义、项目说明、参考卡）版权归 UC Berkeley 与课程作者所有；notes.cs61c.org 页面标注的许可是 CC-BY-NC-ND-4.0（署名—非商业性使用—禁止演绎），因此本文不复述原文段落*，只做中文重述、结构化整理与工程化解读，并逐节标注来源章节号。凡材料未明说、由笔者推断之处均标“推断”；凡课程当学期可能变动之处均标“以课程当学期主页为准”。

1.1 一句话定位

1.2 先修要求（Policies 页原文归纳）

1.3 评分结构（Policies 页 Grading，逐项抄录）

1.4 考试安排（Policies 页 + 主页 Schedule）

1.5 40 讲主题地图（主页 Schedule 表逐条抄录）

1.6 三个项目 + 一个可选项目（主页 + 各项目页）

2.1 动机：为什么从“比特”开始

2.2 基本单位与“k 个东西需要多少位”

2.3 二进制 ↔ 十进制 ↔ 十六进制
2.3.1 权值展开
2.3.2 十六进制是“压缩的二进制”
2.4 “比特可以表示任何东西”
2.5 N 位无符号整数
2.6 无符号溢出：里程表模型
2.6.1 溢出检测（硬件视角）
2.7 本章陷阱
3.1 动机：负数要付出什么代价
3.2 方案一：符号–幅值（Sign–Magnitude）
3.3 方案二：反码（Ones' Complement）
3.4 方案三：补码（Two's Complement）——现代标准
3.4.1 从反码到补码：把负数整体左移一格
3.4.2 形式定义（唯一需要背的公式）
3.4.3 取负：取反加一
3.4.4 C 语言与补码
3.4.5 补码加法：直接用无符号加法器
3.5 符号扩展（Sign Extension）
3.6 四种方案对比（本章总结表）
3.7 本章陷阱清单
4.1 动机：定点数为什么不够
4.2 从科学计数法到二进制规格化
4.3 单精度 IEEE 754 的位布局
4.3.1 为什么指数用偏置编码而不是补码
4.3.2 为什么隐含 1
4.4 特殊值：四类非规格化情形
4.4.1 两个零
4.4.2 无穷
4.4.3 NaN
4.4.4 非规格化数（denorm）与渐进下溢
4.5 溢出 vs 下溢（浮点专有）
4.6 舍入模式
4.7 浮点的三个反直觉性质（考点密集）
4.7.1 加法不满足结合律
4.7.2 整数↔浮点转换会丢精度
4.7.3 精度 ≠ 准确度
4.8 本章陷阱清单
5.1 动机：为什么体系结构课要教 C
5.2 指针、数组、字符串
5.2.1 指针
5.2.2 数组
5.2.3 字符串
5.3 内存布局（C 程序的四段）
5.4 泛型：`void *` 与函数指针
5.4.1 `void *`
5.4.2 函数指针
5.5 抽象数据类型（ADT）
5.6 字节序（Endianness）
5.7 本章陷阱清单
6.1 动机：为什么是 RISC 而不是 x86
6.2 32 个寄存器
6.3 算术与位运算
6.4 访存：只有 load/store 碰内存
6.5 本章陷阱
7.1 六条条件分支
7.2 从 C 到分支：三种控制结构
7.2.1 if–else
7.2.2 while
7.2.3 for
7.3 常用伪指令（会被汇编器展开）
7.4 分支范围限制与“远跳”
7.5 本章陷阱
8.1 动机：为什么不保存寄存器就会崩
8.2 六步调用（谁负责什么）
8.3 调用约定表（必须背）
8.4 栈帧布局（必画图之一）
8.5 递归实例：`factorial`
8.6 caller–saved 与 callee–saved 的实战判断
8.7 本章陷阱
9.1 为什么格式要“少而规整”
9.2 六种格式（green card 原表）
9.3 opcode 速查
9.4 立即数“拼图”：B 型与 J 型为什么乱
9.4.1 B 型重组
9.4.2 J 型重组

9.4.3 U 型
9.5 `li` 的展开：一个必考的边界情况
9.6 寻址模式（只有四种）
9.7 本章陷阱
10.1 四步流水线
10.2 汇编器：目标文件里有什么
10.2.1 汇编器能做什么、不能做什么
10.3 链接器：三件事
10.4 静态链接 vs 动态链接
10.5 装载器做什么
10.6 本章陷阱
11.1 从晶体管到门
11.2 布尔代数与规范形式
11.3 常用组合模块
11.4 加法器与“加/减一体”
11.5 ALU：需要支持哪些运算
11.6 本章陷阱
12.1 寄存器与时钟
12.2 时钟周期由谁决定
12.3 FSM：状态机
12.3.1 例：检测连续三个 1
12.4 本章陷阱
13.1 五步与七个元件
13.2 数据通路的骨架
13.3 逐类指令的路径
13.3.1 R 型（`add rd, rs1, rs2`）
13.3.2 I 型（`addi rd, rs1, imm`）
13.3.3 Load（`lw rd, imm(rs1)`）
13.3.4 Store（`sw rs2, imm(rs1)`）
13.3.5 B 型（`beq rs1, rs2, label`）
13.3.6 J 型（`jal rd, label`）
13.3.7 U 型（`lui rd, imm` / `auipc rd, imm`）
13.3.8 控制信号总表（notes §8.3 Table 1，部分）
13.4 指令时序：为什么单周期“慢得离谱”
13.5 本章陷阱
14.1 五级与四个流水寄存器
14.2 理想 CPI
14.3 结构冒险（Structural Hazard）
14.4 数据冒险（Data Hazard）
14.4.1 解法一：停顿（stall）
14.4.2 解法二：前递 / 旁路（forwarding / bypassing）
14.4.3 解法三：RegFile 的“写后读”（write-then-read）
14.4.4 load-use 冒险：无法完全消除的一拍
14.5 控制冒险（Control Hazard）
14.6 CPI 计算（考试必考）
14.7 本章陷阱
15.1 动机：为什么要层次
15.2 AMAT：唯一的性能公式
15.3 三种映射
15.3.1 直接映射（Direct Mapped）
15.3.2 全相联（Fully Associative）
15.3.3 组相联（N-Way Set Associative）
15.4 替换与写策略
15.5 三类（四类）miss
15.6 优化手段与代价（notes 表 10.3 的结论）
15.7 Cache blocking（分块）
15.8 本章陷阱
16.1 动机：三个问题一起解决
16.2 地址翻译
16.3 页表设计
16.4 TLB：地址翻译的 cache
16.5 三种情形的代价（notes 表 12.1）
16.6 完整层次：一次 load 要过几关
16.7 本章陷阱
17.1 动机：为什么单核不再变快
17.2 Amdahl 定律
17.3 三种并行
17.3.1 ILP（指令级并行）
17.3.2 DLP / SIMD（数据级并行）
17.3.3 TLP（线程级并行）
17.4 共享内存的两个硬问题
17.4.1 数据竞争（data race）
17.4.2 Cache 一致性（cache coherence）

17.4.3 伪共享（false sharing）
17.5 本章陷阱
18.1 依赖关系
18.2 Project 1：`snek`
18.3 Project 2：`CS61Classify`
Part A（函数库）
Part B（矩阵与分类）
18.4 Project 3：`CS61CPU`
Part A（20 分 = 项目 20%）
Part B（80 分 = 项目 80%）
19.1 与同目录其他笔记的分工
19.2 复习清单（按讲次打勾）
19.3 自测题（12 题，附答案）
20.1 课程主页与政策
20.2 讲义页（40 个，正文不可提取）
20.3 课程笔记（notes.cs61c.org，本文的技术依据）
20.4 项目页
20.5 工具与环境
20.6 教材（Policies 页 Reading 一节）
20.7 未能获取或存在不确定性的材料（诚实清单）

第 1 章 课程概览：CS61C 到底在教什么

1.1 一句话定位

CS61C 是一门**“从比特到并行机”的纵向贯通课**：它把 CS61A/61B 建立的“程序”视角，一路向下拆到晶体管、门电路、数据通路、流水线、Cache 与虚拟内存，再向上回到“为什么 C 程序会这样跑”。

Policies 页给出的官方课程描述（Fall 2026 *Official Academic Guide course description*）是：

> The subjects covered in this course include C and assembly language programming, translation of high-level programs into machine language, computer organization, caches, performance measurement, parallelism, CPU design, warehouse-scale computing, and related topics.

把它拆成能力项就是四件事：

能力	对应讲次	你必须能亲手做的事
用比特思考	2、8	手算补码/浮点编码，解释溢出与舍入
用汇编思考	9–14	手写 RISC-V，遵守调用约定，手写递归
用硬件思考	16–25	画单周期数据通路、填控制真值表、算流水线冒险
用系统思考	26–37	算 AMAT、判断命中/缺失、走一遍 TLB 到磁盘的路径

1.2 先修要求（Policies 页原文归纳）

Policies 页 *Prerequisites* 写得很宽松：

> CS 61A and CS 61B (or equivalents) **or** experience with a C-based programming language (e.g. Java, C++).

但实际隐含要求更高。课程默认你已经会：

- 基本数据结构：数组、链表、哈希、递归；
- 至少一门 C 系语言（Java/C++ 亦可），能读懂指针与内存；
- 二进制/十六进制换算（不会的话第 2 讲会非常赶）；
- 命令行、`git`、SSH（Lab 0 会教，但默认你上手快）。

Policies 页明确：这些内容**不会在课上系统重讲**。

1.3 评分结构（Policies 页 *Grading*，逐项抄录）

CS61C 采用**固定分档（fixed scale）**，总分 300 分，不 curve、不调 bin、不做四舍五入以外的处理（0.5 进 1，0.49 舍去）。

项目	分值	说明
Discussion	10	出勤/参与；线下 discussion 需签到，或缺（见下）
Labs	20	Gradescope 自动评分，按通过的测试比例给分
Homework	30	PrairieLearn，按正确率
Project 1 (snek)	10	C
Project 2 (CS61Classify)	20	RISC-V 汇编
Project 3 (CS61CPU)	30	Logisim
Quest	20	CBTF，50 分钟
Midterm	40	CBTF，50 分钟（本学期拆成 Part 1 / Part 2 两次）
Final	120	纸质，闭卷，12/14
合计	**300**	

分档表（Policies 页原表）：

总分	等级	总分	等级
280+	A	240–250	B-
270–280	A-	230–240	C+
260–270	B+	220–230	C

总分	等级	总分	等级
250–260	B	210–220	C-

再往下 D+/D/D-/F 依次 200/190/180/0 起。A+ 由讲师另行决定。

两个必须记住的机制（直接影响复习策略）：

1. **Final Clobber**：若期末的百分比分数高于 Quest 或 Midterm，则用期末折算后的分数**替换**较早那次考试。Policies 页同时警告“don't treat this as a strategy”。此外还有一条 *Final Clobber – Quest*：若期末中**只统计 Quest 覆盖知识点**的那部分子集的百分比高于 Quest 得分，也可替换 Quest。
2. **Lateness**：作业/lab/project 迟交每天扣 **1/3**，不足一天按一天向上取整（晚 1 天 5 秒 = 按 2 天算，扣 2/3）。延期走 **Flextensions**，24 小时内的申请自动批准。

1.4 考试安排（Policies 页 + 主页 Schedule）

考试	形式	窗口	补考窗口
Quest	CBTF，50 分钟	9/16–9/18	9/24–9/25
Midterm Part 1	CBTF，50 分钟	10/28–10/30	11/2–11/3
Midterm Part 2	CBTF，50 分钟	11/9–11/13	11/19–11/20
Final	纸质，3 小时	12/14 11:30AM–2:30PM PT	无（不提供线上期末）

三个 CBTF 考试都**必须自己在 PrairieTest 上预约**；Policies 页写明：忘记在自助预约窗口内预约 = 该次 0 分，且“since all CBTF exams have retakes, there will be no leniency for missing both an exam and its retake”。

1.5 40 讲主题地图（主页 Schedule 表逐条抄录）

这是本笔记的“骨架”。**讲号与讲题均来自 Fall 2026 主页日程表**；`notes` 列是 notes.cs61c.org 里对应的章节（用于查正文）。

讲	讲题（主页原文）	notes 对应章节
1	Intro, Great Ideas	§1.1
2	Number Representation	§1.2–1.4
3	C Intro – Basics	§2.x（c-basics）
4	C Intro – Pointers, Arrays, Strings	§2.x（c-pointers-arrays-strings）
5	C Memory Management	§2.x（c-memory-management）

讲	讲题（主页原文）	notes 对应章节
6	C Generics	§2.x（c-generics）
7	C Abstract Data Types	§2.x + §13（IEC 前缀）
8	Floating Point	§3.1–3.5
9	RISC-V Basics	§4.1–4.2
10	RISC-V Data Transfer	§4.3
11	RISC-V Decision Making	§4.3
12	RISC-V Procedures	§4.4
13	RISC-V Instruction Formats I	§5.1（R/I/S）
14	RISC-V Instruction Formats II	§5.2（B/J/U）
15	Compiler, Assembler, Linker, Loader	§6.1–6.3
16	Intro to SDS	§7.1
17	Combinational Logic	§7.2
18	State	§7.3
19	Blocks and ALU	§7.4
20	RISC-V Single-Cycle Datapath I	§8.1
21	RISC-V Single-Cycle Datapath II	§8.2
22	RISC-V Single-Cycle Control	§8.3
23	RISC-V 5-Stage Pipeline I	§9.1
24	RISC-V 5-Stage Pipeline II – Hazards	§9.2
25	RISC-V 5-Stage Pipeline III – Hazards, Continued	§9.2
26	Caches I: Performance	§10.1
27	Caches II	§10.2
28	Caches III	§10.2–10.3
29	Caches IV	§10.3
30	Parallelism I – Performance, Amdahl's Law	§11.1
31	Parallelism II – SIMD	§11.2
32	Parallelism III – TLP	§11.3
33	Parallelism IV – Concurrency	§11.3

讲	讲题（主页原文）	notes 对应章节
34	Parallelism V – MIMD	§11.3
35	[Guest Lecture] – Parallelism VI	以当学期讲义为准
36	Virtual Memory I	§12.1
37	Virtual Memory II	§12.2
38	[Guest Lecture] – Saving the World with Computing（Kathy Yelick）	以当学期讲义为准
39	[Guest Lecture] – Dave Patterson	以当学期讲义为准
40	Summary, What's Next?	—

> **观察**：40 讲里有 3 讲是 guest lecture（35、38、39），1 讲是总结（40）。真正需要“啃”的技术讲次是 1–37（其中 35 为 guest），硬件部分（16–29）占 14 讲，是课程最重的一段。

1.6 三个项目 + 一个可选项目（主页 + 各项目页）

编号	名称	形式	核心内容	截止（主页）
Project 1	`snek`	个人或 1 名 partner；**必须在 hive 机器上做**	用 C 写一个可玩的贪吃蛇：`game_t` / `snake_t` 结构、堆分配、`fgets` 读盘面、多蛇移动	9/20
Project 2	`CS61Classify`	个人或 1 partner；**本地机器**（不需要 hive）	用 RISC-V 汇编写 MNIST 手写数字分类：`abs` / `relu` / `argmax` / `dot` + Part B 的 `matmul` /读写矩阵/ `classify`	A: 9/25；B: 10/8
Project 3	`CS61CPU`	个人或 1 partner；本地 Logisim	在 Logisim Evolution 里搭 RV32I 单周期 CPU（ALU、RegFile、ImmGen、控制逻辑、load/store、跳转），最后做 2 级流水	A: 10/21；B: 11/5
Project 4	`61kaChow`	可选	主页标为“OPTIONAL: Lab 8 / Project 4”	12/14

三者的能力梯度非常清楚：**P1 考“C 与内存” → P2 考“汇编与调用约定” → P3 考“数据通路与控制”**。它们正好对应期中前后的三段知识。

第 2 章 数字表示 I：比特、进制与无符号数

> 来源：notes §1.1 Introduction、§1.2 Binary/Decimal/Hex、§1.3 Integer Representations（前半）、§1.4 Summary；对应 Lecture 1—2、Homework 1、Discussion 1。

2.1 动机：为什么从“比特”开始

notes §1.1 的开篇问题是：**How do we represent digital data?**

真实世界是模拟的（analog）：声音、图像、电压连续变化。要把它塞进计算机，必须做两步（notes 原文举了 CD 的例子）：

步骤	做什么	notes 的例子
采样 Sample	每隔固定时间问信号“你现在的值是多少”	CD 每秒问 44,100 次
量化 Quantize	把连续幅度“吸附”到最近的刻度	用 16 bit = 2^16 = 65,536 个刻度

量化之后，一个连续的波就变成了一串 16 位样本。**这门课不研究怎么采样（那是信号处理课的事），只研究“给定有限位宽，怎么表示数”**。

2.2 基本单位与“k 个东西需要多少位”

术语	定义	能表示多少
bit（比特）	一个二进制位，取 0 或 1	2
nibble（半字节）	4 bit	16 = 一个十六进制位
byte（字节）	8 bit	256

核心公式：要用比特区分 k 个不同的东西，至少需要

```
N >= ceil(log2(k))          （向上取整）
```

反过来，N 位能表示 2^N 个不同的位模式。

notes §1.4 的例题很典型：`116` 需要多少位?

```
116 = 0b1110100         -> 7 位
log2(116) ≈ 6.85        -> 向上取整 7
```

> **陷阱**：`log2(k)` 恰好是整数时（k = 8, 16, 32…），答案就是 log2(k)，**不要**再多加 1。例如“表示 0..7 共 8 个数”只需 3 位。很多人条件反射写 `floor(log2(k))+1`，在 k 是 2 的幂时会多算一位。

2.3 二进制 ↔ 十进制 ↔ 十六进制

2.3.1 权值展开

一个 N 位二进制串 `b_{N-1} ... b_1 b_0` 的值是：

```
value = sum_{i=0}^{N-1} b_i * 2^i
```

例：

```
0b1011 = 1*8 + 0*4 + 1*2 + 1*1 = 11
0b1111 1111 = 255
```

2.3.2 十六进制是“压缩的二进制”

因为 16 = 2^4，**一个十六进制位正好对应 4 个二进制位（一个 nibble）**。这就是为什么十六进制在体系结构里无处不在：它不改变信息，只是把 4 位缩成 1 个字符。

hex	bin	dec	hex	bin	dec
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

转换口诀（notes §1.2 的做法）：

```
bin -> hex : 从右往左，每 4 位一组，不足补 0，逐组查表
hex -> bin : 每个 hex 字符展开成固定 4 位（不要省略前导 0！）
hex -> dec : 按 16 的幂展开，或先转 bin 再转 dec
dec -> hex : 反复除以 16 取余，或先转 bin
```

例（notes 用的加州金 `0xFDB515`）：

```
0x F D B 5 1 5
  1111 1101 1011 0101 0001 0101
= 1111 1101 1011 0101 0001 0101   (24 bit)
```

这 24 位再按 8 位切成三组，就是 RGB 三个分量：`0xFD`=253（红）、`0xB5`=181（绿）、`0x15`=21（蓝）。

> **陷阱 1**：`0x0F` 展开是 `0000 1111`，不是 `1111`。丢掉前导 0 会直接改变位宽，在后面的符号扩展、cache 字段切分里是致命错误。 **陷阱 2**：C 里 `0x` 是十六进制前缀、`0b` 是二进制前缀（C23 起标准支持 `0b`，GCC 早就支持）；单独的 `0` 前缀是**八进制**，`010` 等于 8，不是 10。

2.4 “比特可以表示任何东西”

notes §1.4 把这条列为第一个 Great Idea。同一个位串，解释方式不同，含义完全不同：

解释方式	8 位 `0100 0001` 的含义
无符号整数	65
补码整数	65
ASCII 字符	`'A'`
位图（bitmask）	第 6 位与第 0 位置 1
指令的一部分	某条 RISC-V 指令的 funct3/opcode 片段

位串本身没有类型，类型是“解释约定”。这条观念在后面会反复出现：cache 里的 tag 是位串，页表项是位串，指令是位串。

2.5 N 位无符号整数

定义（notes §1.3）：

- `0b0...0`（N 个 0）表示 0；
- `0b1...1`（N 个 1）表示 **2^N − 1**；
- 其余按二进制权值展开。

范围：`[0, 2^N − 1]`，共 2^N 个值。

N	范围	常见类型
8	0 … 255	`uint8_t`
16	0 … 65,535	`uint16_t`
32	0 … 4,294,967,295	`uint32_t` / `unsigned int`（通常）
64	0 … 1.8×10^19	`uint64_t`

C 的类型宽度问题：`unsigned int` **不保证**是 32 位。要精确宽度必须用 `<stdint.h>`（有符号 `int8_t`…`int64_t`）与 `<inttypes.h>`（无符号 `uint8_t`…`uint64_t`）。这是 notes §1.3 明确强调的点。

2.6 无符号溢出：里程表模型

notes 用**汽车里程表（odometer）**做类比：里程表从 000000 一路加到你设定的位数上限，然后**回绕到 0**。

定义：**整数溢出（integer overflow）**指算术结果超出了该位宽能表示的范围。

4 位无符号的例子：

```
  1111  (15)
+ 0001  ( 1)
-------
1 0000  -> 丢弃最高进位 -> 0000 (0)      正溢出：15 + 1 = 0

  0000  (0)
- 0001  (1)
-------
  1111  -> 借位回绕          -> 1111 (15)  负溢出：0 - 1 = 15
```

notes 特别强调一条容易混的术语：

> **There is no such thing as integer underflow.**

整数只有“正溢出/负溢出”，**没有 underflow**。underflow 是浮点专有概念（数值太小、掉进 0 附近的空隙），见第 4 章。很多人把“减到负数回绕”叫 underflow，这是错的。

2.6.1 溢出检测（硬件视角）

无符号加法溢出：**最高位的进位输出 carry_out = 1**。

有符号（补码）加法溢出：**进入符号位的进位 ≠ 离开符号位的进位**，等价判定是

```
溢出  <=>  （a 与 b 同号）且（结果的符号位与 a 不同）
```

这两条在 Project 3 的 ALU 里会用到（判断 slt 与 branch 的正确性）。

2.7 本章陷阱

1. **把“负溢出”叫成 underflow** —— 整数没有 underflow，只有正/负溢出。
2. **“k 个值需要 k 位”** —— 是 `ceil(log2 k)` 位；k 恰为 2 的幂时不要多加 1。
3. **十六进制丢前导 0** —— `0x0F` 展开必须是 `0000 1111`。
4. **`010` 当成十进制 10** —— C 里 `0` 前缀是**八进制**，`010` = 8。

5. **以为 `int` 一定是 32 位** —— 用 `<stdint.h>` 的 `int32_t`。
6. **无符号减法“得到负数”** —— 实际回绕成巨大正数。

第 3 章 数字表示 II：符号表示与补码

> 来源：notes §1.3（Sign–Magnitude / Ones' Complement / Two's Complement / Bias Encoding）、§1.4 Summary；对应 Lecture 2、Homework 1、Discussion 1。

3.1 动机：负数要付出什么代价

notes 用一个“国王与经济学家”的故事引出核心权衡：

> **Ain't No Free Lunch.**（天下没有免费的午餐）

要表示负数，就必须**放弃一部分正数范围**。N 位只有 2^N 个位模式，正数、负数、零要在这 2^N 个格子里分。

判定一个符号表示“好不好”，notes 给了四条隐含标准：

1. 零是否唯一?
2. 二进制“里程表”递增时，数值是否也单调递增？（决定比较电路好不好做）
3. 加法能否直接用二进制加法器？（决定 ALU 复杂度）
4. 范围是否对称/够用?

3.2 方案一：符号–幅值（Sign–Magnitude）

规则：**最高位是符号位**（0 正 1 负），其余位是绝对值。

4 位例子：

位串	值	位串	值
0000	+0	1000	−0
0001	+1	1001	−1
0010	+2	1010	−2
…	…	…	…
0111	+7	1111	−7

范围：`[-(2^{N-1}-1), +(2^{N-1}-1)]`，**两个零**。

问题一：**两个零**（`+0` 与 `-0`）。每次比较是否为零都要查两种模式。

问题二（更致命）：**里程表不单调**。递增 `0000 → 0111` 是 0→7，再 +1 变 `1000`，突然跳到 **−0**，然后 `1001` 是 −1、一路到 `1111` 是 −7。也就是说：

```
二进制加 1  ==>  有时是数值 +1，有时是数值 -1
```

这直接毁掉了“加法器复用”的可能。notes 的例子：`+5 + (-5)`：

```
+5 = 0101
-5 = 1101
```

符号不同 → 必须做**绝对值相减**，还要比较哪个绝对值大来决定结果符号。于是 ALU 需要“条件加法/条件减法”两套路径 + 比较器。

结论：notes 称 Sign-Magnitude 是 **straw man**（稻草人）方案——不是不能用，而是不适合通用计算。它的合理用途是**信号处理**：那里经常需要把“符号”和“幅度”解耦处理。

3.3 方案二：反码（Ones' Complement）

规则：负数 = 正数**按位取反**。

4 位例子：

```
+5 = 0101
-5 = 1010      (0101 取反)
```

范围：`[-(2^{N-1}-1), +(2^{N-1}-1)]`，**仍然是两个零**（`0000` = +0，`1111` = −0）。

进步之处：**加法可以直接用二进制加法器**，不需要管符号：

```
  0101  (+5)
+ 1010  (-5)
--------
  1111  -> 解释为 -0
```

数值上 `5 + (-5) = 0`，得到 `-0`，**值是对的**（只要你接受 −0 = 0）。

仍然没解决的问题：两个零。历史上反码真的被用过（notes 提到 “historically, one's complement was used for a while”），后来被补码取代。

计数（notes 的 Quick Check）：N 位反码中

```
零:      2 个
正数:    2^{N-1} - 1 个
负数:    2^{N-1} - 1 个
```

3.4 方案三：补码（Two's Complement）——现代标准

3.4.1 从反码到补码：把负数整体左移一格

notes 的推导非常直观：

```
反码的问题：负数的映射与正数"重叠"，产生两个 0
解法：把负数映射整体左移一格
```

于是 `1111` 从“−0”变成 **−1**，`1000` 从“−7”变成 **−8**。多出来的那个格子给了最负的数。

4 位补码全表：

位串	值	位串	值
0000	0	1000	**−8**
0001	1	1001	−7
0010	2	1010	−6
0011	3	1011	−5
0100	4	1100	−4
0101	5	1101	−3
0110	6	1110	−2
0111	7	1111	**−1**

三个必须背下来的性质：

1. **零唯一**：`0000`。
2. **范围不对称**：`[-2^{N-1}, 2^{N-1}-1]`。负数比正数多一个。
3. **全 1 是 −1；最高位是 1 就是负数**（符号位仍然可读）。

计数：

```
零:      1 个
正数:    2^{N-1} - 1 个
负数:    2^{N-1} 个
```

3.4.2 形式定义（唯一需要背的公式）

n 位补码的值：

```
value = -2^{n-1} * d_{n-1} + sum_{i=0}^{n-2} 2^i * d_i
```

也就是**最高位的权重是负的**，其余位权重照旧为正。

验证 `0b1011`（4 位）：

```
= (1 × -2^3) + (0 × 2^2) + (1 × 2^1) + (1 × 2^0)
= -8 + 0 + 2 + 1
= -5        ✅
```

验证 `0b0101`：

```
= (0 × -8) + (1 × 4) + (0 × 2) + (1 × 1)
= 5         ✅
```

> **这个公式是补码一切性质的源头**：范围不对称、符号扩展、取负、溢出判定都能从它推出来。考试若忘记“取负是取反加一”，用公式现推即可。

3.4.3 取负：取反加一

```
-x  ==  ~x + 1
```

4 位例子：`x = 0101`（+5）

```
~x     = 1010
~x + 1 = 1011   = -5   ✅
```

再取负一次：`~1011 + 1 = 0100 + 1 = 0101`（+5）✅ 回到原值。

> **陷阱（最经典）**：`-2^{N-1}` 取负会**溢出回自己**。4 位时 `x = 1000`（−8），`~x + 1 = 0111 + 1 = 1000`，还是 −8。这不是 bug，是补码范围的必然结果。C 里对 `INT_MIN` 取负是**未定义行为**（UB）。

3.4.4 C 语言与补码

notes §1.3 写明：**补码是 C23 标准规定的有符号整数表示**。在此之前，C 标准允许反码/符号–幅值实现（现实中几乎全是补码）。

所以：

- `int8_t` 范围是 `[-128, 127]`；
- `-128` 的位模式是 `0x80`；
- 有符号右移 `>>` 在 C 里是**实现定义**的（算术右移 or 逻辑右移），但 RISC-V 明确区分 `sra`（算术）与 `srl`（逻辑）。

3.4.5 补码加法：直接用无符号加法器

这是补码最大的工程优势：**同一套加法器，同一套位运算，符号自动正确**。

```
  0101  (+5)
+ 1011  (-5)
--------
1 0000  -> 丢弃进位 -> 0000 (0)      ✅
```

减法用“加负数”实现：`a - b = a + (~b + 1)`。硬件上就是“把 b 取反、进位输入置 1”，这正是 notes §7.4 里 adder/subtractor 的做法（一个 `Sub` 控制位 + 一排 XOR 门）。

3.5 符号扩展（Sign Extension）

把 N 位有符号数放进 M 位（M > N）时，**用符号位填充高位**：

```
4 位 -5  = 1011
8 位 -5  = 1111 1011      (高位补 1)
8 位 +5  = 0000 0101      (高位补 0)
```

规则：

```
sext(x, N -> M) = x 的最高位复制 (M-N) 份放到高位
```

对照：**零扩展（zero extension）**高位补 0，只适用于无符号数。

> **陷阱**：`lw` 与 `lb` / `lh` 的区别就在这里。`lb` 会把读到的字节**符号扩展**到 32 位，`lbu` 做零扩展。读一个值为 `0xFF` 的字节：`lb` 得 −1，`lbu` 得 255。snek 里读盘面字符时全是 ASCII（<128），两者无差别；但一旦用于二进制数据就是 bug 源头。

3.6 四种方案对比（本章总结表）

方案	零的个数	范围（N 位）	加法能否直接用二进制加法器	单调递增	现代用途
无符号	1	`[0, 2^N-1]`	是	是	地址、位运算、计数
符号-幅值	2	`[-(2^{N-1}-1), 2^{N-1}-1]`	**否**（需条件加减）	**否**	信号处理、浮点符号位
反码	2	`[-(2^{N-1}-1), 2^{N-1}-1]`	是	否（回绕处跳变）	历史

方案	零的个数	范围（N 位）	加法能否直接用二进制加法器	单调递增	现代用途
补码	1	`[-2^{N-1}, 2^{N-1}-1]`	**是**	是	**所有现代整数运算**
偏置	1	`[-bias, 2^N-1-bias]`	否（要减 bias）	是	**IEEE 754 指数域**

3.7 本章陷阱清单

1. **把“负溢出”叫成 underflow** —— 整数没有 underflow。
2. **忘记补码范围不对称** —— `int8_t` 是 `[-128, 127]` 不是 `[-127, 127]`。
3. **对 `INT_MIN` 取负** —— 溢出回自己，C 里是 UB。
4. **符号扩展写成零扩展** —— `lw` vs `lbu`、`addi` 的 12 位立即数一定符号扩展。
5. **以为补码加法要特殊处理符号** —— 不需要，和无符号加法完全一样。
6. **十六进制丢掉前导 0** —— `0x0F` ≠ `0xF` 当位宽重要时。
7. **比较时不看 signed/unsigned** —— `blt` vs `bltu`、`slt` vs `sltu` 是两组指令，选错就错。

第 4 章 浮点：IEEE 754 深入

> 来源：notes §3.1 Motivation（定点）→ §3.2 Normalized Numbers → §3.3 Practice → §3.4 Special Numbers → §3.5 More Discussion；对应 Lecture 8、Homework 3、Discussion 3。

4.1 动机：定点数为什么不够

定点（fixed point）把二进制点钉死在某一位，比如 Q16.16 就是“高 16 位整数、低 16 位小数”。问题：**动态范围太小**。

需求	定点 Q16.16 能表示吗
表示 0.1640625	可以
表示 6.02×10^23（阿伏伽德罗常数）	不行，溢出
表示 1.6×10^−19（元电荷）	不行，全部舍成 0

物理量跨越几十个数量级，**必须让二进制点的位置随数浮动**——这就是 floating point 的字面含义。IEEE 754 的做法借自科学计数法。

4.2 从科学计数法到二进制规格化

十进制科学计数法：`0.1640625 = 1.640625 × 10^-1`

术语	含义	本例
radix（基数）	底数	10
mantissa（尾数）	有效数字	1.640625
exponent（指数）	基数的幂	−1

规格化（normalized）：尾数小数点左边恰好一位非零数字。规格化保证“同一个数只有一种写法”。

二进制里只有一种非零数字 `1`，所以**规格化二进制尾数一定长成 `1.xxxxx`**：

```
0.1640625 = 0.0010101_2 = 1.0101_2 × 2^-3
```

这一步推出两个关键结论：

1. 32 位必须**切成两个域**：一个存尾数、一个存指数。
2. 既然尾数一定以 `1.` 开头，这个 1 **可以不存**（implicit leading one），白赚 1 位精度。

4.3 单精度 IEEE 754 的位布局

```
 31  30           23  22                                0
+----+---------------+-----------------------------------+
| s  |   exponent    |           significand             |
| 1b |     8 b       |              23 b                 |
+----+---------------+-----------------------------------+
```

域	位宽	含义（规格化数）
`s`	1	0 正 1 负
`exponent`	8	**偏置编码**，真值 = 域值 − 127
`significand`	23	小数部分，真尾数 = **1 + significand**

求值公式（**本章唯一必须背的公式**）：

```
value = (-1)^s × (1 + significand) × 2^(exponent - 127)
```

双精度（`double`，C 的 64 位）：

域	位宽	bias
s	1	—
exponent	11	**1023**
significand	52	—

`double` 的规格化范围约 `±[2.2×10^−308, 1.8×10^308]`（notes 给的是“about 2.0×10^−308 to 2.0×10^308”）。

4.3.1 为什么指数用偏置编码而不是补码

notes §3.2 给了明确理由：**为了让浮点数能用整数比较器排序**。

若指数用补码，负数指数的位模式以 1 开头，在**无符号**整数比较下会显得“更大”，排序就乱了。用偏置后，指数域越大代表指数越大，于是：

> 同符号的两个浮点数，直接按位当作无符号整数比较，大小关系正确

这就是为什么浮点比较可以很快——甚至可以用整数流水线。

4.3.2 为什么隐含 1

因为规格化尾数必以 1 开头。省下这一位后，23 位的域实际表达 **24 位精度**（notes 原话：“the precision is 24 bits, though we only use 23 bits”）。

4.4 特殊值：四类非规格化情形

指数域的 0 与 255 被**保留**，用来编码非规格化数、零、无穷、NaN：

指数域	significand	含义
`0000 0000` (0)	全 0	**±0**
`0000 0000` (0)	非 0	**非规格化数**（denorm）
`0000 0001` – `1111 1110` (1–254)	任意	**规格化数**（隐含前导 1）
`1111 1111` (255)	全 0	**±∞**
`1111 1111` (255)	非 0	**NaN**

4.4.1 两个零

```
+0 : s=0, exp=00000000, sig=0...0
−0 : s=1, exp=00000000, sig=0...0
```

为什么需要 −0？notes 的解释是科学计算需要：`1/(+0) = +∞`、`1/(−0) = −∞`，以及极限方向。注意**`+0 == −0` 在比较时为真**，尽管位模式不同。

4.4.2 无穷

```
+∞ : s=0, exp=11111111, sig=0...0
−∞ : s=1, exp=11111111, sig=0...0
```

作用：溢出时“优雅地”给出 ±∞，而不是像整数那样回绕成负数。`x / ±0` 得 ±∞（x ≠ 0 时）。

4.4.3 NaN

```
NaN : exp=11111111, sig ≠ 0
```

产生原因：非法运算，如 `sqrt(−4)`、`0/0`、`∞ − ∞`。

污染性（notes 原话）：`op(NaN, x) = NaN`。任何与 NaN 的运算都得 NaN。判定必须用 `x != x`（NaN 是唯一不等于自己的值），不能用 `==`。

4.4.4 非规格化数（denorm）与渐进下溢

规格化数的下界是 `2^−126`（指数域 = 1）。而规格化数在 0 附近的**步长**是 `2^−149`（因为尾数最低位权重是 `2^−23 × 2^−126`）：

```
最小规格化数:      1.000...0 × 2^−126 = 2^−126
次小规格化数:      1.000...1 × 2^−126 = 2^−126 + 2^−149
```

问题：`2^−126` 到 0 之间有一段**巨大的空隙**——从 0 直接跳到 `2^−126`，中间什么都没有。这会让“很小的数相减”突然变成 0，破坏精度。

解法：**非规格化数**，格式为

```
value = (−1)^s × (0.significand) × 2^−126
```

即：**不加隐含 1**，指数**固定为 −126**（不是 −127）。于是步长在全区间统一为 `2^−149`，实现**渐进下溢（gradual underflow）**：

```
0 ... 2^−149 ... 2×2^−149 ... ... 2^−126（最小规格化）... 2^−125 ...
|<------- denorm 区（步长 2^−149）------>|<-- 规格化区 -->
```

量	值
最小正 denorm	`2^−149` ≈ 1.4×10^−45
最小正规格化数	`2^−126` ≈ 1.2×10^−38

量	值
最大有限值	`(2 − 2^−23) × 2^127` ≈ 3.4×10^38

> **陷阱**：denorm 的隐含指数是 **−126**，不是 −127。这是最高频的编码题错误。记忆法：“denorm 与最小规格化数共享同一个指数 −126，只是尾数不补 1，于是能一路平滑走到 0”。

4.5 溢出 vs 下溢（浮点专有）

现象	定义	结果
Overflow	数值绝对值太大	±∞
Underflow	数值绝对值太小，落进 denorm/0 区	denorm 或 ±0

对比整数：整数只有 overflow（回绕），**没有 underflow**；浮点两者都有，且**都不会“回绕”**——这是浮点比整数“安全”的地方。

4.6 舍入模式

浮点硬件内部保留**额外两位精度**（notes：“carries two extra bits of precision”，即 guard / round 位）再舍入。四种模式：

模式	规则	例（2.001 / 1.999 / −2.001）
向 +∞ 舍入	永远“向上”	2.001→3；−2.001→−2
向 −∞ 舍入	永远“向下”	1.999→1；−1.999→−2
截断（向 0）	直接丢低位	—
向偶数舍入（unbiased，默认）	就近舍入，正好居中时取偶	2.4→2；2.6→3；**2.5→2；3.5→4**

默认是**向最近偶数舍入（round to nearest, ties to even）**。为什么“ties to even”而不是“ties up”？因为长期看一半向上、一半向下，**误差不累积**（unbiased）。

> **陷阱**：`2.5 → 2`、`3.5 → 4`，不是“四舍五入”。判断方法：看要保留的最后一位，若为奇数则进位使其变偶，若为偶数则舍去。

4.7 浮点的三个反直觉性质（考点密集）

4.7.1 加法不满足结合律

notes §3.5 的例子：

```
x = -1.5e38, y = 1.5e38, z = 1.0

(x + y) + z = 0.0 + 1.0 = 1.0
x + (y + z) = -1.5e38 + 1.5e38 = 0.0     ← y+z 中的 1.0 被"吞掉"
```

原因：`1.5e38` 的步长远大于 1.0，`1.5e38 + 1.0` 舍入回 `1.5e38`。

工程含义：并行归约（reduction）的结果依赖求和顺序；这也是为什么浮点并行程序“每次跑结果略有不同”。

4.7.2 整数↔浮点转换会丢精度

```
int  i = 16777217;          // 2^24 + 1
float f = (float) i;        // 舍入成 16777216.0f (2^24)
```

因为 `float` 只有 24 位有效精度，`2^24 + 1` 无法表示，会“snap”到最近的偶数。

反方向：

```
int j = (int) 1.5;          // C 用截断 -> 1，不是 2
```

所以下面两段代码**不保证**打印 true（notes §3.5 原文示例）：

```
int i = ...;
if (i == (int)((float) i)) printf("true\n");     // 大整数会失败

float f = ...;
if (f == (float)((int) f)) printf("true\n");     // 有小数会失败
```

4.7.3 精度 ≠ 准确度

概念	定义
Precision（精度）	用多少位表示一个值
Accuracy（准确度）	表示值与真实值的差

notes 强调：**高精度允许高准确度，但不保证**。一个 64 位浮点可以很精确地表示一个错误的中间结果。

4.8 本章陷阱清单

1. denorm 的隐含指数写成 −127（应为 **−126**）。
2. 忘记 denorm **不补隐含 1**。
3. 把 `exp=0, sig=0` 当成 denorm（那是 **±0**）。
4. 把 `exp=255, sig≠0` 当成 ∞（那是 **NaN**）。
5. 舍入模式记成“四舍五入”（应为 **ties-to-even**）。
6. 认为 `+0 == -0` 为假（实际为**真**）。
7. 认为浮点加法可结合（**不可**）。
8. 认为 `(int)f` 会四舍五入（实际**截断**）。
9. 忘记指数域是**偏置**而非补码。
10. 双精度 bias 写成 127（应为 **1023**）。

第 5 章 C 语言与内存模型

> 来源：notes §2.x（c-basics / c-pointers-arrays-strings / c-memory-management / c-generics / c-odds-and-ends）；对应 Lecture 3—7、Lab 1—2、Project 1。

5.1 动机：为什么体系结构课要教 C

因为 C **是“可移植的汇编”**：它的抽象层次刚好高到能写程序，低到能看见内存。

C 的特性	暴露的硬件事实
指针就是地址	内存是一个巨大的一维字节数组
数组名退化为指针	数组 = 连续内存 + 首地址
`malloc/free`	堆是显式管理的
局部变量在栈上	函数调用使用栈帧
没有边界检查	越界就是踩别人的内存
`int` 宽度依赖平台	必须用 `<stdint.h>` 固定宽度

对比 Java：Java 有 GC、有边界检查、对象在堆、引用不可运算。notes 的 *C vs. Java* 一节专门做这个对照。

5.2 指针、数组、字符串

5.2.1 指针

```
int x = 42;
int *p = &x;      // p 存放 x 的地址
*p = 43;          // 通过 p 改写 x
```

三条铁律：

1. 指针有**类型**，决定“解引用时读几个字节、怎么解释”。
2. 指针算术按**所指类型的大小**缩放：`p + 1` 前进 `sizeof(*p)` 字节。
3. `NULL` 是地址 0，解引用它是未定义行为（绝大多数平台上 segfault）。

5.2.2 数组

```
int a[10];
// a[i]  ==  *(a + i)
// &a[i] ==  a + i
```

数组名不是指针变量，它在表达式中**退化（decay）**成指向首元素的指针；`sizeof(a)` 仍是整个数组大小（在定义所在作用域内）。

二维数组 `int m[3][4]` 在内存里是**行主序（row-major）**的连续 12 个 int：

```
m[0][0] m[0][1] m[0][2] m[0][3] m[1][0] ... m[2][3]
   ^ 偏移 0                        ^ 偏移 (1*4+0)*4 字节
```

通用公式（Project 2 的 `matmul` 直接用）：

```
&m[i][j] = base + (i * ncols + j) * sizeof(elem)
```

5.2.3 字符串

C 字符串是**以 `\0` 结尾的 char 数组**：

```
char s[] = "hi";        // 占 3 字节: 'h' 'i' '\0'
strlen(s) == 2          // 不含 '\0'
sizeof(s) == 3          // 含 '\0'
```

函数	作用	坑
`strlen`	数到 `\0`	O(n)，循环里别反复调
`strcpy`	拷贝含 `\0`	**不检查目标容量**

函数	作用	坑
`strncpy`	限长拷贝	可能**不补** `\0`
`strcmp`	比较	返回 0 表示相等（反直觉）
`strchr`	找字符	返回指针或 NULL

Project 1 的 `LOAD_board` 明确要求用 **`fgets`** 读行，并提示 `strchr` 可能有用（用来找 `\n`）。

5.3 内存布局（C 程序的四段）

```
高地址
+----------------------+
|        stack         |  局部变量、返回地址、保存的寄存器
|         |            |  向低地址增长 ↓
+----------------------+
|          v           |
|                      |
|        heap          |  malloc/new 分配
|         ^            |  向高地址增长 ↑
+----------------------+
|    static / .bss     |  全局变量、static 变量
+----------------------+
|     text (code)      |  机器指令（只读）
+----------------------+
低地址
```

段	存放	生命周期	谁管理
text	指令	整个进程	loader
static/data	全局、`static`、字符串字面量	整个进程	编译器/loader
heap	`malloc`	手动 `free`	程序员
stack	局部变量、参数、返回地址	函数调用期间	编译器（prologue/epilogue）

Project 1 的 Task 1 提示里直接问：“Which part of memory (code, static, stack, heap) should you store the new game in?”——答案是 **heap**（因为要 `free_game` 释放，且大小运行时决定）。

5.4 泛型：`void *` 与函数指针

C 没有模板，靠两样东西实现“泛型”：

5.4.1 `void *`

```
void *memcpy(void *dst, const void *src, size_t n);
```

`void *` 是“无类型地址”，可以隐式转成任何对象指针，但**不能解引用**（不知道大小）。

5.4.2 函数指针

```
int (*cmp)(const void *, const void *);
qsort(arr, n, sizeof(int), cmp);
```

`int (*add_food)(game_t* game)` 读法：`add_food` 是一个指针，指向“接受 `game_t*`、返回 `int`”的函数。

Project 1 的 Task 4.5 就用到了这个：`update_game` 的签名是

```
void update_game(game_t *game, int (*add_food)(game_t *game));
```

notes 的原文解释值得抄下来：

> The `int (*add_food)(game_t* game)` argument is a function pointer, which means that `add_food` is a pointer to **the code section of memory**.

这句话把“函数指针指向 text 段”这件事说透了。

5.5 抽象数据类型（ADT）

ADT = **接口（`.h`）+ 实现（`.c`）分离**。

```
/* game.h —— 只暴露不完整类型 */
typedef struct game game_t;
game_t *create_default_game(void);
void free_game(game_t *game);
```

```
/* game.c —— 完整定义 */
struct game {
    unsigned int num_rows;
    char **board;
    unsigned int num_snakes;
    snake_t *snakes;
};
```

好处：调用方看不到字段布局 → 实现可改；坏处：不能栈上分配（大小未知），只能指针 + 堆。

Project 1 的 `game.h` 就是这种风格（不过它把 struct 完整定义放在头文件里，因为作业需要你直接操作字段）。

5.6 字节序（Endianness）

notes 的 *endianness* 一节：

```
int x = 0x12345678;
```

机器	内存低地址 → 高地址
小端（little-endian）	`78 56 34 12`
大端（big-endian）	`12 34 56 78`

RISC-V 与 x86 都是小端。含义：

- 一个 32 位字的**最低有效字节**放在**最低地址**；
- `char *p = (char*)&x;` 读到的 `*p` 是 `0x78`；
- cache 的 block offset 从低地址侧开始切。

> **陷阱**：把 `int` 的首字节当成最高字节（大端思维）——在 snek 的盘面解析、Project 2 的矩阵数据读写里都会踩。

5.7 本章陷阱清单

1. `sizeof(数组)` 在函数参数里变成 `sizeof(指针)` = 8。
2. 返回局部数组的指针（栈已失效）。
3. `strcpy` 目标缓冲区太小 → 溢出。
4. `malloc` 后忘记 `free`（Project 1 用 valgrind 查）。
5. `free` 后继续用（use-after-free）。
6. 混淆 `a[i]` 与 `*a + i`。
7. 以为 `char` 默认有符号（**实现定义**，通常有符号）。
8. 在 `printf` 里用错格式符（`%d` 配 `long` → UB）。
9. 忘记 `\0` 导致字符串拼接越界。
10. 认为 `int` 一定是 32 位（用 `<stdint.h>`）。

第 6 章 RISC-V 基础：寄存器、算术与访存

> 来源：notes §4.1 Elements / §4.2 RISC-V ISA / §4.3 Arithmetic & Bitwise / §4.4 Load-Store（rv-intro、rv-data-transfer）；green card。对应 Lecture 9—10、Lab 3。

6.1 动机：为什么是 RISC 而不是 x86

维度	RISC（RISC-V）	CISC（x86）
指令长度	**固定 32 位**（RV32I）	变长 1—15 字节
指令数量	少（RV32I 约 40 条）	上千条
寻址模式	少（寄存器、imm、base+disp、PC 相对）	多
访存	**只有 load/store 能碰内存**	算术指令可直接读内存
译码	简单（字段位置固定）	复杂（微码）

RISC 的核心赌注：**简单指令 + 流水线 > 复杂指令**。第 15—16 章会证明这个赌注怎么兑现。

6.2 32 个寄存器

编号	ABI 名	用途	保存者
x0	`zero`	**恒为 0**（写入被丢弃）	—
x1	`ra`	返回地址	Caller
x2	`sp`	栈指针	Callee
x3	`gp`	全局指针（本课 out of scope）	—
x4	`tp`	线程指针（本课 out of scope）	—
x5—7	`t0-t2`	临时寄存器	Caller
x8	`s0/fp`	保存寄存器 / 帧指针	Callee
x9	`s1`	保存寄存器	Callee
x10—11	`a0-a1`	参数 / **返回值**	Caller
x12—17	`a2-a7`	参数	Caller
x18—27	`s2-s11`	保存寄存器	Callee
x28—31	`t3-t6`	临时寄存器	Caller

`x0` 的妙用：`addi x0, x0, 0` = **nop**；`beq rs1, x0, label` = **beqz**；`jal x0, label` = **j**。

6.3 算术与位运算

指令	含义	格式
`add rd, rs1, rs2`	rd = rs1 + rs2	R
`sub rd, rs1, rs2`	rd = rs1 − rs2	R
`addi rd, rs1, imm`	rd = rs1 + sext(imm)	I
`and/or/xor`	按位	R
`andi/ori/xori`	按位立即数	I
`sll/srl/sra rd, rs1, rs2`	左移/逻辑右移/**算术**右移	R
`slli/srli/srai`	移位立即数（5 位）	I*
`slt/sltu`	小于置 1（有/无符号）	R
`slti/sltiu`	立即数版本	I
`mul/mulh/mulhu`	乘法（M 扩展）	R

三个必须理解的点：

1. **没有 `subi`**。要减立即数就加负数：`addi rd, rs1, -5`。
2. **`sra` vs `srl`**：算术右移补符号位（负数保持负），逻辑右移补 0。`-8 >> 1` 用 `sra` 得 −4，用 `srl` 得一个巨大正数。
3. **`slt` vs `sltu`**：同一个位模式，一个有符号解释一个无符号解释。`0xFFFFFFFF` 在 `slt` 下是 −1，在 `sltu` 下是 42 亿。

```
例：x5 = 0xFFFFFFFF, x6 = 1
slt  x7, x5, x6   ->  x7 = 1   (-1 < 1)
sltu x7, x5, x6   ->  x7 = 0   (4294967295 > 1)
```

6.4 访存：只有 load/store 碰内存

指令	宽度	扩展方式
`lb`	8 位	**符号扩展**到 32
`lbu`	8 位	**零扩展**
`lh` / `lhu`	16 位	符号 / 零扩展
`lw`	32 位	—

指令	宽度	扩展方式
`sb` / `sh` / `sw`	8/16/32	只写低位

唯一的地址模式（base + displacement）：

```
addr = R[rs1] + sext(imm12)

lw  rd, imm(rs1)      # rd = M[rs1 + imm]
sw  rs2, imm(rs1)     # M[rs1 + imm] = rs2
```

例：

```
int a[10];
a[3] = a[2] + 1;
```

```
# s0 = &a[0]
lw   t0, 8(s0)        # a[2]  (2*4 = 8)
addi t0, t0, 1
sw   t0, 12(s0)       # a[3]  (3*4 = 12)
```

内存是小端：`sw` 一个 `0x12345678` 后，最低地址那个字节是 `0x78`。

> **陷阱**：RISC-V 允许非对齐访问在部分实现上触发异常；教学模拟器 Venus 通常容忍。写 `lw` 时地址应当是 4 的倍数。

6.5 本章陷阱

1. **给 `slli` 传大于 31 的立即数** —— 只有 5 位，超出是未定义。
2. **`addi` 的 12 位立即数当无符号** —— 它一定符号扩展（`0xFFF` = −1）。
3. **用 `srl` 处理负数** —— 会补 0 变成巨大正数，应该用 `sra`。
4. **`lw` 地址不是 4 的倍数** —— 部分实现会触发异常。
5. **以为算术指令能直接读内存** —— RISC-V 只有 load/store 能碰内存。
6. **把 `x0` 当普通寄存器写** —— 写入被丢弃，值恒为 0。

第 7 章 RISC-V 决策与循环

> 来源：notes §4.3 Decision Making（branches、loops）；对应 Lecture 11、Homework 4。

7.1 六条条件分支

指令	条件	有/无符号
`beq rs1, rs2, label`	rs1 == rs2	—
`bne`	rs1 != rs2	—
`blt`	rs1 < rs2	有符号
`bltu`	rs1 < rs2	无符号
`bge`	rs1 >= rs2	有符号
`bgeu`	rs1 >= rs2	无符号

没有 `bgt` / `ble`：把两个操作数交换即可（`bgt a,b` → `blt b,a`）。

分支是 **PC 相对（PC-relative）** 的：目标地址 = 当前 PC + 偏移。B 型偏移是 **13 位、以半字为单位**，范围 `±2^10` 条指令（即 ±4 KiB 字节）。

7.2 从 C 到分支：三种控制结构

7.2.1 if-else

```
if (a < b) { x = 1; } else { x = 2; }
```

```
    bge  a0, a1, else     # 条件取反，跳过 then
    li   t0, 1
    j    done
else:
    li   t0, 2
done:
```

关键技巧：条件取反。汇编里通常先写“不成立就跳走”，这样 then 块可以顺序落下。

7.2.2 while

```
while (i < n) { s += a[i]; i++; }
```

```
loop:
    bge  t0, a1, done     # i >= n -> 退出
    slli t2, t0, 2
    add  t2, a0, t2
    lw   t3, 0(t2)
    add  s0, s0, t3
    addi t0, t0, 1
    j    loop
done:
```

7.2.3 for

`for (init; cond; step) body;` 与 `while` 完全等价：

```
init
loop:  if (!cond) goto done
       body
       step
       goto loop
done:
```

7.3 常用伪指令（会被汇编器展开）

伪指令	展开	说明
`beqz rs, L`	`beq rs, x0, L`	与 0 比较
`bnez rs, L`	`bne rs, x0, L`	
`j L`	`jal x0, L`	无条件跳
`jr rs`	`jalr x0, rs, 0`	寄存器跳
`ret`	`jalr x0, ra, 0`	返回
`mv rd, rs`	`addi rd, rs, 0`	复制
`li rd, imm`	`lui`（+`addi`）或 `addi`	装常数
`la rd, label`	`auipc` + `addi`	取地址
`neg rd, rs`	`sub rd, x0, rs`	取负
`not rd, rs`	`xori rd, rs, -1`	按位取反
`nop`	`addi x0, x0, 0`	空操作

> **陷阱**：伪指令只能在**汇编源码**里写。Project 3 的 CPU 只认**真指令**，所以你在 Logisim 里看到的机器码一定是展开后的形式。Project 2 用 Venus 时可以用伪指令（Venus 会自动展开）。

7.4 分支范围限制与“远跳”

B 型只能跳 `±2^10` 条指令。要跳更远，用**两条指令组合**：

```
    bne  a0, x0, next     # 条件相反，跳过紧跟的 j
    j    far              # J 型，范围 ±2^18 条指令
next:
```

若连 `±2^18` 都不够，就用绝对地址：

```
    auipc ra, %pcrel_hi(far)     # 取 PC 高 20 位
    jalr  x0, ra, %pcrel_lo(far) # 加低 12 位并跳
```

7.5 本章陷阱

1. 用 `blt` 比较无符号数（应 `bltu`）。
2. 忘记分支是 PC 相对的，手算偏移时漏掉“以半字为单位”。
3. `beq` / `bne` 之外的分支写反（`blt` vs `bge` 是互补，不是相反）。
4. `li` 大常数时忘记 `addi` 的符号扩展（见第 9 章）。
5. 循环变量用 `s` 寄存器却没在序言保存（见第 8 章）。

第 8 章 RISC-V 过程调用与调用约定

> 来源：notes §4.4 Procedures（rv-jumps-stack、rv-calling-conv、examples）；green card。对应 Lecture 12、Lab 4、Discussion 5、Project 2。

8.1 动机：为什么不保存寄存器就会崩

notes 给了一个能一句话说明问题的例子：

```
int mult(int x, int y)      { return x * y; }
int sum_square(int x, int y){ return mult(x,x) + mult(y,y); }
```

`sum_square` 用 `jal ra, mult` 调用两次。问题在于：**`ra` 只有一个**。第一次调用把 `ra` 覆盖成“返回 `sum_square` 的地址”，第二次调用又覆盖一次——但这次的返回值仍然正确，因为两次都返回同一个地方。真正的坑在于**递归**：

```
int factorial(int n) {
    if (n == 1) return 1;
    return n * factorial(n-1);      // 调用后还要用 n
}
```

`factorial(2)` 把 `1` 放进 `a0`、`jal` 调用 `factorial(1)`；返回后 `a0` 里是返回值 `1`，但**原来那个 `2` 已经没了**——它被 callee 覆盖了。

结论：**必须有一个地方保存“我还要用的东西”**。那个地方就是**栈**。

8.2 六步调用（谁负责什么）

步	谁	做什么
1	Caller	准备参数：放入 `a0–a7`（多余的放栈）；保存 caller-saved 寄存器（若返回后还要用）
2	Caller	`jal ra, func` 转移控制
3	Callee	**序言（prologue）**：`sp -= frame_size`；保存要用的 `s` 寄存器与 `ra`（若自己还会调用别人）
4	Callee	执行函数体
5	Callee	**尾声（epilogue）**：返回值放 `a0`（必要时 `a1`）；恢复 `s` 寄存器与 `ra`；`sp += frame_size`
6	Callee	`jr ra`（= `jalr x0, ra, 0`）返回

8.3 调用约定表（必须背）

寄存器	别名	跨调用是否保留	谁保存
x0	zero	恒 0	—
x1	ra	**否**	Caller
x2	sp	**是**	**Callee**
x3–4	gp, tp	不适用（out of scope）	—
x5–7	t0–t2	**否**	Caller
x8–9	s0–s1	**是**	**Callee**
x10–11	a0–a1	**否**	Caller
x12–17	a2–a7	**否**	Caller

寄存器	别名	跨调用是否保留	谁保存
x18—27	s2—s11	**是**	**Callee**
x28—31	t3—t6	**否**	Caller

记忆法：

```
t = temporary = 临时 = caller 自己看着办（volatile）
s = saved     = 存档 = callee 必须原样还回来
a = argument  = 参数/返回值 = 也是 volatile
ra = 返回地址 = 也是 volatile（所以递归必须自己存）
sp = 栈指针   = 必须由 callee 恢复（否则调用者栈就废了）
```

关键理解（notes 原话）：调用约定**不是“禁止使用”某些寄存器**。任何时刻你都能读写全部 31 个寄存器；约定只规定**返回时必须恢复哪些**。这是一个双向契约：

- Caller 期望：返回后 `s*` 与 `sp` 不变，`t*` / `a*` / `ra` 可能变了。
- Callee 承诺：我随便改 `t*`，但 `s*` 我会还原。

8.4 栈帧布局（必画图之一）

```
高地址
+----------------------------+  <- 调用者的栈
|   caller 的局部变量         |
+----------------------------+
|   参数 8, 9, ...（若有）    |  <- 第 9 个及以后的参数由 caller 压栈
+----------------------------+
|   ra（若 callee 还会调用）  |  <- callee 的帧开始
|   s0 .. sN（若用到）        |
|   局部数组 / 结构体         |
+----------------------------+  <- sp（callee 序言里减过）
|   ...                      |
低地址
```

序言/尾声的标准写法：

```
func:
    addi sp, sp, -16        # 分配 16 字节帧
    sw   ra, 12(sp)         # 存返回地址
    sw   s0, 8(sp)          # 存要用的 s 寄存器
    sw   s1, 4(sp)
    ...
    lw   s1, 4(sp)          # 尾声：逆序恢复
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    jr   ra
```

> **为什么帧大小是 16 的倍数？** RISC-V psABI 要求 `sp` 保持 **16 字节对齐**（`sp mod 16 == 0`）。分配奇数大小会破坏对齐，某些库函数（如需要 16 字节对齐的 SIMD 操作）会崩。Project 2 的 calling convention checker 会检查这一点。

8.5 递归实例：`factorial`

```
# int factorial(int n)   -- a0 = n, 返回 a0
factorial:
    addi sp, sp, -8
    sw   ra, 4(sp)          # 必须存 ra！否则递归返回地址丢失
    sw   a0, 0(sp)          # 必须存 n！否则返回后 n 被覆盖
    li   t0, 1
    bne  a0, t0, recurse
    li   a0, 1              # base case
    j    done
recurse:
    addi a0, a0, -1
    jal  ra, factorial      # 递归
    lw   t1, 0(sp)          # 取回 n
    mul  a0, a0, t1         # n * factorial(n-1)
done:
    lw   ra, 4(sp)
    addi sp, sp, 8
    jr   ra
```

两个 `sw` 缺一不可：`ra` 缺了 → 返回地址丢失，跳到随机地址；`a0`（n）缺了 → 乘法用错值。这就是 notes 里 Quick Check 想问的答案。

8.6 caller-saved 与 callee-saved 的实战判断

写一个函数时，问自己三个问题：

1. **我会调用别人吗？** 会 → 必须存 `ra`。

2. **调用前算出来的值，调用后还要用吗？** 要 → 要么放进 `s` 寄存器（并保存它），要么压栈。
3. **我用到的 `s` 寄存器超过可用数量吗？** 超过 → 溢出的压栈。

Project 2 的 `matmul` 就是典型：它要循环调用 `dot`，而 `dot` 会破坏 `t0–t6` 与 `a1–a7`。所以 `matmul` 的循环变量、指针、维度**必须**放在 `s` 寄存器或栈上。

8.7 本章陷阱

1. 递归函数忘记存 `ra`。
2. 调用后继续用 `t*` / `a*` 里调用前的值。
3. 改了 `s*` 却没恢复 → 调用者的变量静默损坏（最难查的 bug）。
4. `sp` 减了却没加回（或反过来）。
5. 帧大小不是 16 的倍数。
6. 返回值放错寄存器（应 `a0`）。
7. 参数超过 8 个时忘记用栈传。

第 9 章 六种指令格式与寻址模式

> 来源：notes §5.1 Instruction Formats I（R/I/S）、§5.2 Instruction Formats II（B/J/U）、§4.2 green card；对应 Lecture 13–14、Homework 5。

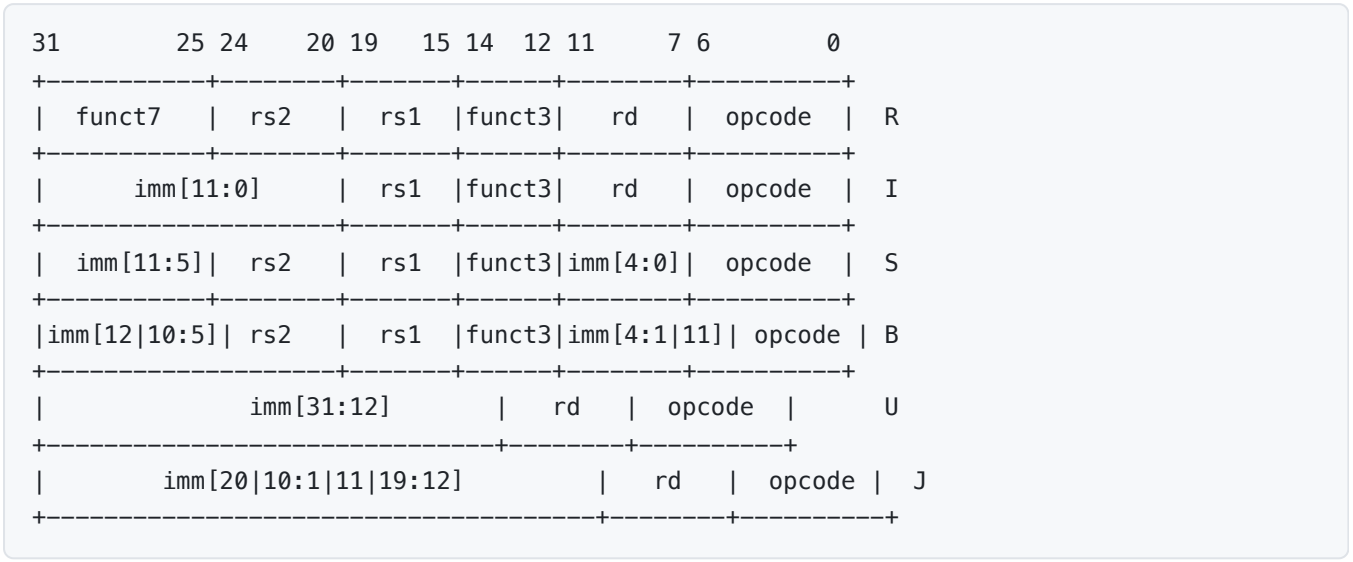
9.1 为什么格式要“少而规整”

RISC 的设计目标：**译码器要简单到能用几根线搞定**。所以：

- 所有指令 32 位；
- `opcode` 永远在 `[6:0]`；
- `rd` 永远在 `[11:7]`；
- `funct3` 永远在 `[14:12]`；
- `rs1` 永远在 `[19:15]`；
- `rs2` 永远在 `[24:20]`；
- `funct7` 永远在 `[31:25]`。

只有**立即数**的位置会变——这就是六种格式的唯一区别。

9.2 六种格式（green card 原表）



类型	用途	立即数位宽	是否符号扩展
R	寄存器–寄存器运算	无	—
I	立即数运算、load、`jalr`	12	是
S	store	12	是
B	条件分支	13（含隐含 0 位）	是
U	`lui`、`auipc`	20（放在高 20 位）	—
J	`jal`	21（含隐含 0 位）	是

9.3 opcode 速查

指令组	opcode（二进制）	十六进制
R 型	`011 0011`	0x33
I 型算术	`001 0011`	0x13
Load	`000 0011`	0x03
Store (S)	`010 0011`	0x23
Branch (B)	`110 0011`	0x63
`jalr`	`110 0111`	0x67
`jal` (J)	`110 1111`	0x6F
`lui` (U)	`011 0111`	0x37

指令组	opcode（二进制）	十六进制
`auipc` (U)	`001 0111`	0x17
`ecall/ebreak`	`111 0011`	0x73

9.4 立即数“拼图”：B 型与 J 型为什么乱

立即数的位被“打散”不是随意为之，而是为了**让符号位永远落在 `inst[31]`**，与 I/S 型保持一致——这样硬件只要一根线就能取到符号位。

9.4.1 B 型重组

```
inst[31]    -> imm[12]     (符号位)
inst[30:25] -> imm[10:5]
inst[11:8]  -> imm[4:1]
inst[7]     -> imm[11]
imm[0]      <- 恒 0        (半字对齐)
```

Verilog 式写法：

```
imm = {{20{inst[31]}}, inst[7], inst[30:25], inst[11:8], 1'b0};
```

9.4.2 J 型重组

```
inst[31]    -> imm[20]     (符号位)
inst[30:21] -> imm[10:1]
inst[20]    -> imm[11]
inst[19:12] -> imm[19:12]
imm[0]      <- 恒 0
```

```
imm = {{12{inst[31]}}, inst[19:12], inst[20], inst[30:21], 1'b0};
```

9.4.3 U 型

```
imm[31:12] = inst[31:12]
imm[11:0]  = 0
```

`lui rd, immu` 语义就是 `rd = immu << 12`。

> **陷阱**：B/J 型的 `imm[0]` 永远是 0（指令必须 4 字节对齐，偏移至少是 2 字节的倍数）。少写这一位，偏移就错了一倍。

9.5 `li` 的展开：一个必考的边界情况

`li rd, imm` 分三种情况：

```
-2048 <= imm <= 2047   ->  addi rd, x0, imm
否则                    ->  lui rd, hi20 ; addi rd, rd, lo12
```

`hi20` / `lo12` 的算法与**符号扩展**有关：

```
hi20 = (imm + 0x800) >> 12;      // 加 0x800 是"进位修正"
lo12 = imm & 0xFFF;              // 有符号的 12 位
```

例 1：`li x10, 0x87654321`

```
lui  x10, 0x87654        # x10 = 0x87654000
addi x10, x10, 0x321     # x10 = 0x87654321   ✅
```

例 2（边界）：`li x10, 0xB0BACAFE`

```
错:  lui x10, 0xB0BAC ; addi x10, x10, 0xAFE
     0xAFE 的符号位是 1 -> 被符号扩展成 0xFFFFFAFE
     结果 = 0xB0BAC000 + 0xFFFFFAFE = 0xB0BABAFE   ❌ 差 0x1000

对:  lui x10, 0xB0BAD ; addi x10, x10, 0xAFE
```

规律：**若 `lo12` 的最高位是 1（即 ≥ 0x800），`hi20` 要加 1。**

9.6 寻址模式（只有四种）

模式	形式	例	用于
立即数	`imm`	`addi rd, rs1, imm`	常数运算
寄存器	`rs1, rs2`	`add rd, rs1, rs2`	寄存器运算
基址+偏移	`imm(rs1)`	`lw rd, 8(sp)`	load/store、`jalr`
PC 相对	`label`	`beq rs1, rs2, L` / `jal ra, L`	分支、跳转

绝对地址不是独立模式，而是用两条指令拼出来：`lui` + `addi`（`la`）或 `auipc` + `jalr`。

9.7 本章陷阱

1. B/J 型忘记隐含的 `imm[0]=0`。

2. `li` 大常数时没有做 `hi20` 进位修正。
3. I 型立即数当无符号处理（它**一定**符号扩展，`0xFFF` 是 −1 不是 4095）。
4. `slli` / `srli` 的立即数只有 **5 位**（0–31），超出是未定义。
5. `jalr` 的立即数是 I 型（12 位有符号）。
6. `lui` 的立即数是“高 20 位”，不是“低 20 位”。
7. `sw` 的立即数被拆成 `[11:5]` 与 `[4:0]` 两块，抄错顺序。

第 10 章 工具链：编译 → 汇编 → 链接 → 装载（CALL）

> 来源：notes §6.1 CALL Overview / §6.2 Assembler: Object File / §6.3 Linker: Resolve References；对应 Lecture 15、Homework 5、Discussion 6。

10.1 四步流水线

```
foo.c ──[compiler]──> foo.s ──[assembler]──> foo.o ──[linker]──> a.out ──[loader]──> 运行
      （C → 汇编）            （汇编 → 机器码模块）      （合并模块）          （装入内存）
```

步	输入	输出	关键动作
Compile	`foo.c`	`foo.s`	生成**可能含伪指令**的汇编
Assemble	`foo.s`	`foo.o`	展开伪指令；产生**目标文件**
Link	`foo.o` + `lib.o`	`a.out`	合并段；**解析引用**；重定位
Load	`a.out`	（运行）	建地址空间、拷段、初始化寄存器、跳 `_start`

10.2 汇编器：目标文件里有什么

`.o` 是**二进制**，包含六块：

部分	内容	给谁用
Object file header	各部件的**大小与位置**（目录）	linker
Text segment	机器码	linker
Data segment	静态数据的二进制	linker

部分	内容	给谁用
Symbol table	本文件里可被外部引用的**标签**（函数、全局变量）	linker、gdb
Relocation table	“稍后要修”的地址清单（**待办列表**）	linker
Debugging info	行号、变量名	gdb

10.2.1 汇编器能做什么、不能做什么

引用类型	汇编器能解析吗	原因
同模块内的 PC **相对**分支/跳转	✅	知道本模块内标签的相对位置
外部函数（如 `printf`）	❌	不知道别的模块在哪
静态数据的绝对地址（`la`）	❌	数据段可能被 linker 搬到别处

因此汇编器要**两遍扫描**（two passes）：第一遍记录所有标签位置进符号表，第二遍才能算出**前向引用**（forward reference）的偏移。只扫一遍的话，“跳到后面某个标签”的偏移是未知的。

偏移的计算单位是半字（2 字节）——这就是为什么 B 型偏移要左移一位。

10.3 链接器：三件事

1. **拼 text**：把所有 `.o` 的 text 段按顺序接起来。
2. **拼 data**：把所有 `.o` 的 data 段接起来，**放在 text 之后**。
3. **解析引用（resolve references）**：把 relocation table 上的占位符替换成最终地址。

哪些指令需要被“编辑”：

指令	为什么
J 型外部跳转（`jal` 到库函数）	目标地址未知
`auipc` + `jalr` / `auipc` + `addi`	同上（绝对/PC 相对高 20 位）
访问 `.data` 的 `lw` / `sw`	数据段位置未知
B 型分支	**不需要**！模块内相对偏移已由汇编器算好

RV32 的 linker 约定：**text 段从 `0x10000` 开始**（这是 Venus / 教学链接器的约定；真实 Linux 上由 linker script 决定）。

10.4 静态链接 vs 动态链接

维度	静态（`.a`）	动态（`.so` / DLL）
可执行文件大小	**大**（库被整个拷进去）	小
运行时开销	无	有（加载时链接）
库升级	必须**重新编译**程序	换 `.so` 即生效
部署	单文件，简单	需要多个文件共存
内存占用	每个程序各一份	多程序**共享**同一份

notes 的判断：动态链接增加了编译器/linker/OS 的复杂度，但**收益大于成本**。

10.5 装载器做什么

1. 读 `a.out` 头，得到 text/data 大小；
2. 建立**新的地址空间**（足够容纳 text、data、stack）；
3. 把指令与数据从文件拷进去；
4. 把命令行参数拷到栈上；
5. 清空寄存器，`sp` 指向栈顶；
6. 跳到启动例程（`_start`），它再把参数取出、调 `main`；
7. `main` 返回 → 调 `exit` 系统调用终止。

第 2 步“建立地址空间”就是第 19 章虚拟内存的入口。

10.6 本章陷阱

1. 以为汇编器能解析所有标签（外部符号不能）。
2. 以为 B 型分支需要 linker 重定位（不需要）。
3. 混淆 `.o`（可重定位）与 `a.out`（可执行）。
4. 认为静态链接“更快”（启动更快，但体积大、升级难）。
5. 忽略 `text` 在 `data` **之前**这一布局约定。

第 11 章 数字逻辑 I：组合逻辑与 ALU

> 来源：notes §7.1 SDS / §7.2 Combinational Logic（boolean-algebra、cl-design）/ §7.4 Blocks（adder-subtractor、alu）；对应 Lecture 16—17、19、Lab 5。

11.1 从晶体管到门

层	抽象	关键事实
晶体管	MOS 开关	CMOS 用 PMOS+NMOS 互补，静态几乎不耗电
门	AND/OR/NOT/NAND/NOR/XOR	NAND 是“通用门”
组合逻辑	无记忆，输出只依赖当前输入	有**传播延迟**
时序逻辑	有状态（寄存器）	由时钟同步

SDS（Synchronous Digital Systems） 的两条纪律：

1. 所有寄存器由**同一个时钟**驱动；
2. 组合逻辑的延迟必须**小于一个时钟周期**。

11.2 布尔代数与规范形式

基本定律（做题最常用的四条）：

```
交换/结合/分配      A(B+C) = AB + AC
德摩根               ~(A·B) = ~A + ~B      ~(A+B) = ~A·~B
吸收                 A + AB = A
互补                 A + ~A = 1   A·~A = 0
```

积之和（Sum of Products, SOP）：对真值表里每个输出为 1 的行写一个乘积项，再全部 OR 起来。这是“从真值表直接到电路”的机械方法。

例：`F = A·~B + ~A·B`（XOR）：

```
A B | F
0 0 | 0
0 1 | 1     -> ~A·B
1 0 | 1     -> A·~B
1 1 | 0
```

无关项（don't care）：输出标 `*` 的行可以任意取 0 或 1，用来化简。控制逻辑真值表里大量使用（见第 14 章）。

11.3 常用组合模块

模块	功能	输入→输出
2-to-1 MUX	二选一	`sel ? B : A`
4-to-1 MUX	四选一	2 位 `sel`
Decoder	n→2^n 独热	用于选中寄存器/ROM 行
Priority Encoder	输出最高位为 1 的下标	Project 3 用它选 ROM 行
Adder	加法	见下
ALU	多功能	见下

11.4 加法器与“加/减一体”

半加器：`S = A xor B`，`Cout = A and B`。 **全加器**：三个输入（A, B, Cin），`S = A xor B xor Cin`，`Cout = majority(A,B,Cin)`。 **行波进位加法器（RCA）**：32 个全加器串联，延迟与位宽成正比（慢但简单）。

加/减一体（notes §7.4 的 adder-subtractor）：把 B 的每一位先与 `Sub` 异或，同时 `Cin = Sub`：

```
Sub = 0 : B xor 0 = B,  Cin = 0  ->  A + B
Sub = 1 : B xor 1 = ~B, Cin = 1  ->  A + ~B + 1 = A - B    ✅
```

一个控制位就换来减法，这就是补码的工程红利。

11.5 ALU：需要支持哪些运算

Project 3A 的 Task 1 要求实现 notes 里 **ALU 章节 Table 2** 定义的“项目专用 ALU 运算与对应 `ALUSel` 编码”；**`add` 已经预先做好**，且 Part A 的测试只使用 Table 2 里已定义的编码（未定义编码不必处理）。**唯一可确证的具体编码是 `add = 0b0000`**（spec 原文：“you can hard-code ... `ALUSel = 0b0000`”）。

因此下表的“运算”列是有依据的功能清单，**`ALUSel` 列的具体编码请以 starter 电路 / notes Table 2 为准**（本文不臆造这些数值）：

#	运算	被谁使用
1	ADD	`add`、`addi`、load/store 地址、分支目标
2	SUB	`sub`、分支比较（相减看是否为 0）
3	AND	`and`、`andi`
4	OR	`or`、`ori`

#	运算	被谁使用
5	XOR	`xor`、`xori`
6	SLL	`sll`、`slli`
7	SRL	`srl`、`srli`
8	SRA	`sra`、`srai`
9	SLT	有符号小于
10	SLTU	无符号小于
11	MUL / MULH / MULHU	M 扩展（Part B）
12	直通 B（copyB）	**`lui`**：把立即数原样送到 `rd`（**推断**：spec 提示“Part A 里做过但没用上的某个 ALU 操作”正好给 `lui` 用）

> **说明**：`0b0000 = add` 来自 Project 3A spec 原文；其余编号在公开页面上没有给出，属于 starter 电路内部约定。做题时**照抄 starter 的 Table 2**，不要用本文或其他博客的编号。

`slt` 的实现：`A − B`，然后看**符号位与溢出**：

```
slt 结果 = (A_sign != B_sign) ? A_sign : (A−B)_sign_after_overflow_correction
```

教学实现通常直接扩展一位做减法，或复用 `A−B` 的符号位并处理溢出。

11.6 本章陷阱

1. 把 MUX 的 `sel` 极性接反（`0` / `1` 输入对调）。
2. `sra` 与 `srl` 混用。
3. `slt` 与 `sltu` 混用。
4. 忘了 ALU 也要输出 `zero` 标志（分支用）。
5. RCA 的进位链方向接反。

第 12 章 数字逻辑 II：时序、FSM 与流水线电路

> 来源：notes §7.3 State（register、timing-synchronous-system、fsm、pipelining）；对应 Lecture 18、Discussion 7。

12.1 寄存器与时钟

寄存器（register） = 一组 D 触发器，在**时钟上升沿**采样输入并保持。

```
        +-----------+
D ----->|           |
        |  Register |-----> Q
CLK --->|           |
        +-----------+
        上升沿：Q <- D
```

时序三大约束：

约束	含义	违反后果
Setup time	数据必须在时钟沿**之前**稳定一段时间	采到抖动的值
Hold time	数据必须在时钟沿**之后**再稳定一段时间	采到下一个值
Clock-to-Q delay	时钟沿到 Q 稳定输出的延迟	计入路径延迟

12.2 时钟周期由谁决定

```
          ┌──── 一个时钟周期 T ────┐
CLK  ─────┘                        └─────

寄存器A --[组合逻辑延迟 t_cl]--> 寄存器B
   |<-- t_cq -->|<---- t_cl ---->|<-- t_setup -->|

T >= t_cq + t_cl + t_setup
```

关键结论：时钟频率由**最长的那条组合路径（critical path）**决定，不是平均值。

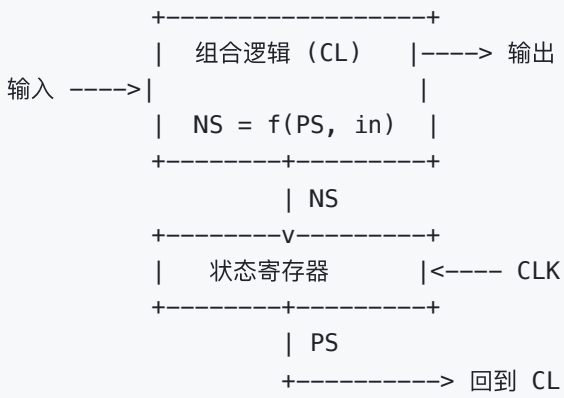
```
f_max = 1 / (t_cq + t_cl,max + t_setup)
```

这条公式是第 13—15 章“为什么单周期慢、流水线快”的全部物理基础。

12.3 FSM：状态机

五要素：输入集合、输出集合、状态集合、状态转移函数、输出函数。

硬件结构（永远是这个形状）：



12.3.1 例：检测连续三个 1

状态定义：`S0`（还没见到 1）、`S1`（见到 1 个）、`S2`（见到 2 个）。

现态 PS	输入	次态 NS	输出
00 (S0)	0	00	0
00 (S0)	1	01	0
01 (S1)	0	00	0
01 (S1)	1	10	0
10 (S2)	0	00	0
10 (S2)	1	00	**1**

注意 `S2` 收到 1 后回到 `S0`（不是停在 S2）——因为“三个 1”已经消费掉了，重叠检测需要额外的 S3 状态才能支持（notes 的版本不支持重叠）。

自环（self-loop）：`S0` 收到 0 保持 `S0`，这是“输入没有改变状态但仍可能有输出变化”的弧。

12.4 本章陷阱

1. **把时钟周期按“平均延迟”估算** —— 必须按**最长组合路径**算。
2. **FSM 漏掉某个输入的自环** —— 未定义转移会让电路进入非法状态。
3. **状态编码不唯一导致输出错** —— 先定编码再写真值表。
4. **忘记寄存器有 setup/hold 约束** —— 组合逻辑再快也不能违反时序。
5. **把组合逻辑输出直接接回自己的输入** —— 形成组合环，Logisim 会报振荡。
6. **复位后状态未初始化** —— 依赖 Logisim 复位为 0 时要显式确认。

第 13 章 单周期数据通路、控制与指令时序

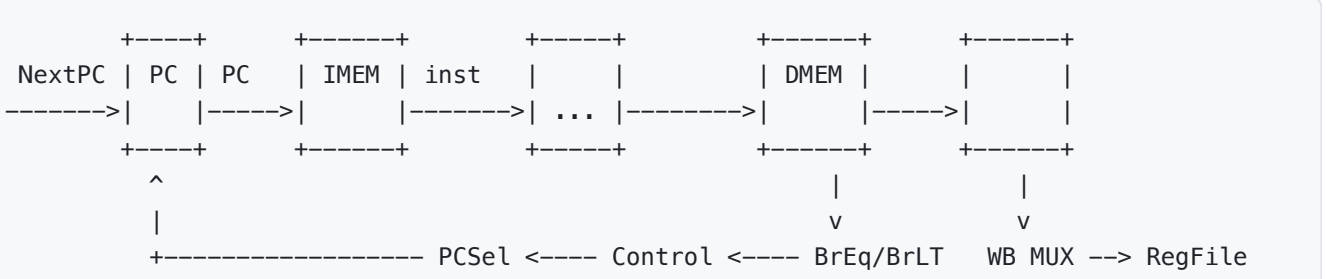
> 来源：notes §8.1 Datapath（elements、r-type、i-type、load-store、b-type、jump、u-type）、§8.2、§8.3 Control & Instruction Timing；对应 Lecture 20—22、Homework 6、Project 3。

13.1 五步与七个元件

任何 RISC-V 指令都走这五步：**IF → ID → EX → MEM → WB**。

元件	作用	关键端口
PC	程序计数器	输入 NextPC，输出 PC
IMEM	指令存储器	地址→指令
RegFile	32×32 寄存器堆	2 读口（`ReadIndex1/2`）+ 1 写口（`WriteIndex/WriteData`）+ `RegWEn`
ImmGen	立即数生成	指令 + `ImmSel` → 32 位立即数
ALU	运算	A、B、`ALUSel` → `ALUResult`、`zero`
DMEM	数据存储器	`MemAddress`、`MemWriteData`、`MemRW`、`MemWriteMask`
Branch Comp	分支比较	rs1、rs2、`BrUn` → `BrEq`、`BrLT`

13.2 数据通路的骨架



注意两条“反向”数据流（notes 特别指出）：

1. **WB 阶段写回 RegFile**（数据从右往左回到中间）；
2. **MEM 阶段决定 NextPC**（控制信号从右往左回到最左边）。

这两条反向路径正是第 16 章**数据冒险**与**控制冒险**的物理根源。

13.3 逐类指令的路径

13.3.1 R 型（`add rd, rs1, rs2`）

```
PC -> IMEM -> inst
inst[19:15] -> RegFile.ReadIndex1 -> rdata1 -> ALU.A
inst[24:20] -> RegFile.ReadIndex2 -> rdata2 -> ALU.B     (BSel = Reg)
inst[11:7]  -> RegFile.WriteIndex
ALUResult   -> RegFile.WriteData                          (WBSel = ALU)
RegWEn = 1
```

控制信号：`ASel=Reg, BSel=Reg, ALUSel=Add, RegWEn=1, MemRW=Read, WBSel=ALU, PCSel=+4`。

13.3.2 I 型（`addi rd, rs1, imm`）

与 R 型唯一区别：**B 口来自 ImmGen**（`BSel = Imm`），`ImmSel = I`。

13.3.3 Load（`lw rd, imm(rs1)`）

```
ALU: rs1 + imm -> MemAddress
DMEM 读出 -> WBSel = Mem -> rd
```

`ASel=Reg, BSel=Imm, ALUSel=Add, MemRW=Read, RegWEn=1, WBSel=Mem`。

13.3.4 Store（`sw rs2, imm(rs1)`）

```
ALU: rs1 + imm -> MemAddress
rs2 -> MemWriteData
MemRW = Write, RegWEn = 0     <- 不写寄存器!
```

13.3.5 B 型（`beq rs1, rs2, label`）

```
ALU: PC + imm -> 分支目标
Branch Comp: rs1 vs rs2 -> BrEq / BrLT
Control: PCSel = (条件成立) ? ALU : PC+4
```

`ASel = PC, BSel = Imm, ALUSel = Add, RegWEn = 0, MemRW = Read`。

13.3.6 J 型（`jal rd, label`）

```
rd <- PC + 4        (WBSel = PC+4)
PC <- PC + imm      (PCSel = ALU)
```

13.3.7 U 型（`lui rd, imm` / `auipc rd, imm`）

```
lui   : rd <- imm            (ALUSel = 直通B（推断）, ASel = *, BSel = Imm)
auipc : rd <- PC + imm       (ASel = PC, BSel = Imm, ALUSel = Add)
```

13.3.8 控制信号总表（notes §8.3 Table 1，部分）

指令	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWEn	WBSel
`add`	+4	*	*	Reg	Reg	Add	Read	1	ALU
`sub`	+4	*	*	Reg	Reg	Sub	Read	1	ALU
`addi`	+4	I	*	Reg	Imm	Add	Read	1	ALU
`lw`	+4	I	*	Reg	Imm	Add	Read	1	Mem
`sw`	+4	S	*	Reg	Imm	Add	**Write**	**0**	*
`beq`	见表下	B	*	**PC**	Imm	Add	Read	0	*
`bne`	见表下	B	*	PC	Imm	Add	Read	0	*
`blt`	见表下	B	**0**	PC	Imm	Add	Read	0	*
`bltu`	见表下	B	**1**	PC	Imm	Add	Read	0	*
`jalr`	ALU	I	*	Reg	Imm	Add	Read	1	**PC+4**
`jal`	ALU	J	*	PC	Imm	Add	Read	1	**PC+4**
`auipc`	+4	U	*	PC	Imm	Add	Read	1	ALU

分支的 `PCSel` 由 `BrEq` / `BrLT` 决定（`*` = don't care）：

指令	分支?	BrEq	BrLT	PCSel
`beq`	不跳	0	*	+4
`beq`	跳	1	*	ALU
`bne`	不跳	1	*	+4
`bne`	跳	0	*	ALU
`blt` / `bltu`	跳	*	1	ALU

> **两处必须记住的理由**：`sw` 的 `RegWEn = 0`（store 不写寄存器）；分支的 `ASel = PC`（目标 = PC + imm，不是 rs1 + imm）。 **注**：表中 `ImmSel` / `ALUSel` / `WBSel` 用的是 notes 的**语义名**（I/S/B/U/J、Add/Sub、ALU/Mem/PC+4），不是二进制编码——二进制编码由你 Project 3 的 ROM 表自行定义，只要“编码与电路一致”即可（Project 3B Task 5.2 原文允许任意一致编码）。

13.4 指令时序：为什么单周期“慢得离谱”

单周期的时钟周期必须容纳**最慢的那条指令**：

指令	经过的元件	相对延迟（示意）
`add`	PC→IMEM→Reg→ALU→Reg	短
`lw`	PC→IMEM→Reg→ALU→**DMEM**→Reg	**最长**
`beq`	PC→IMEM→Reg→ALU→BranchComp→PC	长
`jal`	PC→IMEM→Reg→ALU→PC	中

所以：

```
T_single = t_cq + t_IMEM + t_Reg + t_ALU + t_DMEM + t_setup
```

浪费在哪：`add` 根本不用 DMEM，却被迫等 DMEM 的时间。这就是流水线的动机——让不同指令在不同阶段并行，把“等待”填满。

13.5 本章陷阱

1. `sw` 把 `RegWEn` 置 1（store 不写寄存器）。
2. `beq` 的 `ASel` 接成 `Reg`（必须是 PC，因为要算 PC+imm）。
3. `lui` 用 `Add` 而不是 `copyB`，导致结果变成 `rs1+imm`。
4. `jal` 的 `WBSel` 接成 `ALU`（应为 PC+4）。
5. RegFile 的读写口接反（`rs2` 接到 `ReadIndex1`）。
6. 忘了 `x0` 的写保护。
7. DMEM 的 `MemWriteMask` 没接（部分写会写坏整个字）。

第 14 章 五级流水线与三类冒险

> 来源：notes §9.1 Throughput/Latency/5-Stage Pipeline、§9.2 Hazards（structural/data/control）；对应 Lecture 23—25、Homework 7、Lab 6、Project 3B Task 10。

14.1 五级与四个流水寄存器

级	名称	做什么
1	**IF**	从 IMEM 取指，算 PC+4
2	**ID**	译码、读 RegFile、生成立即数
3	**EX**	ALU 运算、分支比较
4	**MEM**	读/写 DMEM
5	**WB**	写回 RegFile

四个流水寄存器（**按插入位置命名**）：

```
IF/ID : PC, inst
ID/EX : PC, rdata1, rdata2, imm, inst(部分)
EX/MEM: PC, ALUResult, rdata2(store 数据), inst(部分)
MEM/WB: PC+4, DMEM 读出值, ALUResult, inst(部分)
```

> **关键纪律**：一个信号在哪个阶段产生、在哪个阶段被用，必须**逐个阶段核对**。Project 3B Task 10.1 的建议就是这个：“go through your entire circuit and make sure that you specify which stage's value you want to use for any values that are different between stages”。

14.2 理想 CPI

```
理想流水线：CPI ≈ 1（每周期完成一条）
单周期      ：CPI = 1，但时钟周期长得多
```

加速比（级数均衡时）：

```
Speedup = 流水线级数 / (1 + 流水线开销)
```

级数越多收益越递减，因为寄存器开销占比上升，且冒险代价变大。

14.3 结构冒险（Structural Hazard）

定义：两条指令同一周期争用**同一个硬件资源**。

典型：单端口内存，IF 要取指、MEM 要读数据 → 冲突。

解法	代价
拆成 IMEM + DMEM（**本课做法**）	多一个存储器
加 bank / 双端口	硬件复杂
停顿（stall）	性能损失

CS61C 的数据通路**默认 IMEM 与 DMEM 分离**，所以结构冒险在作业里通常不出现。

14.4 数据冒险（Data Hazard）

定义：后一条指令要读的值，还没被前一条指令写回（RAW，read-after-write）。

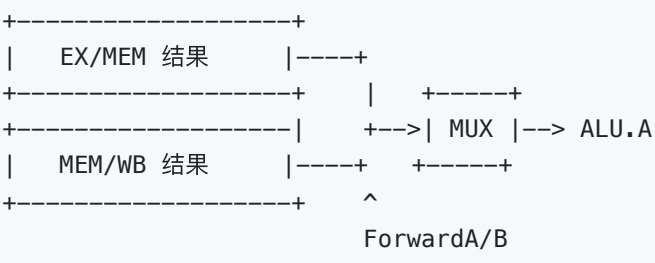
```
add s0, t0, t1      # 周期 5 才写 s0
sub t2, s0, t0      # 周期 3 就读 s0 -> 读到旧值
```

14.4.1 解法一：停顿（stall）

插入 3 个 nop 等到 WB 完成。简单但慢。

14.4.2 解法二：前递 / 旁路（forwarding / bypassing）

不去等写回，而是把 EX/MEM 或 MEM/WB 里的结果直接送到 ALU 输入。



两条前递路径（Project 3 之外的课程要求，作业里要会画）：

路径	从	到	解决
EX/MEM → EX	上一条指令的 ALU 结果	当前指令的 ALU 输入	相邻 ALU 指令
MEM/WB → EX	上上条指令的结果	当前指令的 ALU 输入	隔一条的依赖

14.4.3 解法三：RegFile 的“写后读”（write-then-read）

即使有前递，`add` → `sw` 这种“写寄存器 → 下一周期读寄存器”也可能踩到边界。RISC-V 流水线**假设 RegFile 支持写后读**：

```
WB 阶段：在时钟周期的**前半**（下降沿）写
ID 阶段：在时钟周期的**后半**（上升沿前）读
```

这样同一周期内“先写后读”，省掉一次停顿。

> **陷阱**：这个假设不是所有高频设计都成立。notes 明确提醒：“Always check processor design assumptions beforehand, particularly when answering homework questions.”

14.4.4 load-use 冒险：无法完全消除的一拍

```
lw  t0, 0(s0)       # 数据在 MEM 阶段末尾才拿到
add t1, t0, t2      # 下一条就要用 -> 前递也来不及
```

原因：`lw` 的数据在 **MEM 结束**才可用，而 `add` 的 EX 在**同一周期**就需要它。所以**必须停 1 拍**。

```
lw   IF ID EX MEM WB
add     IF ID -- EX MEM WB        <- 1 个 bubble
```

14.5 控制冒险（Control Hazard）

定义：取到的下一条指令可能不是该执行的（分支/跳转结果未知）。

分支在 **MEM 阶段**才确定（EX 算目标、比较器出结果，MEM 才更新 PC），此时已取进 3 条指令。

方案	做法	代价
1. 遇分支就停	一直停到 MEM	每次都停 3 拍
2. **假设不跳（predict not taken）**	继续取顺序指令；若真跳则 flush	跳转时停 3 拍，不跳时 0
3. 分支预测	动态预测器	预测错才 flush

本课的五级流水线采用方案 2（notes 原话：“The 5-Stage Pipeline resolves control hazards by assuming that branches are not taken”）。

```
beq  t0, t1, Label
sub  t2, s0, t0      # 若跳转，这 3 条要 flush 成 nop
or   t6, a0, t0
xor  t5, t1, s0
Label: sw s0, 8(t3)
```

flush 的实现：把 IF/ID 里的指令替换成 `nop`（`addi x0, x0, 0` = `0x00000013`），并且**不写寄存器、不写内存**。

14.6 CPI 计算（考试必考）

```
CPI = 1 + 停顿周期数 / 指令数
```

常见题型：

场景	停顿
理想流水线	0
每个 load-use 冒险	+1
每个 taken 分支（假设不跳）	+3（或 +2，取决于分支在哪个阶段解析）
每个 taken 分支（分支在 EX 解析）	+2

性能公式合成（Iron Law）：

```
time/program = (instructions/program) × CPI × (time/cycle)
```

`CPI < 1` 只有在**超标量（superscalar）**下才可能（每周期发射多条）。

14.7 本章陷阱

1. 把流水线当成“降低单条指令延迟”（它降低的是**吞吐时间**）。
2. 认为前递能解决 load-use（**不能**，必须停 1 拍）。
3. flush 时忘了同时清 `RegWEn` 与 `MemRW`（会把垃圾写进寄存器/内存）。
4. 分支目标算错（`PC+imm` 用的是**分支指令自己的 PC**，不是 PC+4）。
5. 认为流水线寄存器可以省略（省了就变成“多条指令共用一套线”，功能错）。
6. CPI 算成“平均延迟”。

第 15 章 Cache 与存储层次

> 来源：notes §10.1 Cache Performance（memory-hierarchy、amat）、§10.2（fully-associative / direct-mapped / set-associative）、§10.3（optimizations、blocking）；对应 Lecture 26–29、Homework 8、Discussion 10。

15.1 动机：为什么要层次

层级	典型容量	典型延迟	由谁管理
寄存器	几十×32 位	< 1 周期	编译器
L1 cache	32–128 KiB	~1–4 周期	硬件
L2 / L3	数百 KiB – 数十 MiB	~10–40 周期	硬件
DRAM	GiB	~100–300 周期	硬件 + OS
磁盘/SSD	TiB	~10^5–10^7 周期	OS

层次能成立，全靠**局部性（locality）**：

类型	定义	例子
时间局部性（temporal）	刚访问过的，很快还会访问	循环变量 `i`、`sum`
空间局部性（spatial）	访问了 `x`，很快会访问 `x+1`	遍历数组、顺序取指

15.2 AMAT：唯一的性能公式

```
AMAT = Hit Time + Miss Rate × Miss Penalty
```

术语	定义
Hit rate	命中比例
Miss rate	1 – hit rate
Hit time	访问 cache 并判定命中的时间
Miss penalty	从下一层取回并替换的时间

两级 cache 的递归形式（考试高频）：

```
AMAT = L1_HitTime + L1_MissRate × (L2_HitTime + L2_MissRate × L2_MissPenalty)
```

课程的两个假设（notes §10.1 明确列出）：

1. 一次 miss 的总时间 = **hit time + miss penalty**（不是只有 miss penalty）；
2. **下层 cache 的 miss rate 是“上层 miss 中再次 miss 的比例”**，不是全局比例。

例题（notes 原文数据）：L1 hit 1 周期、miss 5%、penalty 200 周期：

```
AMAT = 1 + 0.05×200 = 11 周期
```

加一个 L2（hit 5 周期、miss 15%、penalty 200）：

```
AMAT = 1 + 0.05×(5 + 0.15×200) = 1 + 0.05×35 = 2.75 周期
```

加了 L2 快了 4 倍。这就是为什么真实 CPU 有三层 cache。

15.3 三种映射

15.3.1 直接映射（Direct Mapped）

规则：**一个块只能放在唯一位置**。

```
index = block_address mod (number_of_blocks)
```

地址切分：

```
+------------------+-----------+----------+
|      tag         |   index   |  offset  |
+------------------+-----------+----------+
   剩下的高位          选哪一块     块内第几字节
```

字段	位数
offset	`log2(block_size)`
index	`log2(number_of_blocks)`
tag	`地址位宽 - index - offset`

例（notes 用图）：16 B cache、4 B block、12 位地址 →

```
offset = log2(4)  = 2
index  = log2(4)  = 2        (4 个块)
tag    = 12-2-2   = 8
```

替换策略：只有唯一候选 → **没有选择**（LRU/FIFO/随机全都“不适用”）。 **写策略**：write-through 与 write-back **都可以**。

15.3.2 全相联（Fully Associative）

规则：**块可以放在任何位置 → 没有 index**。

```
+------------------------+----------+
|          tag           |  offset  |
+------------------------+----------+
```

- 需要与**所有** tag 并行比较（硬件昂贵）；
- 替换策略：LRU / FIFO / 随机；
- 容量小的时候才用（如 TLB）。

15.3.3 组相联（N-Way Set Associative）

规则：先映射到**一个组**，组内可放**任意 way**。

```
index = block_address mod (number_of_sets)
```

概念	关系
总块数	`capacity / block_size`
组数	`总块数 / N`
`N`（相联度）	每组几个 way

例：32 B、2-way、4 B block → 8 块 → **4 组 × 2 way**。

需要 valid bit 吗? 需要，**每个块一个**（不是每组一个）——因为要判断“这一 way 里有没有有效数据”。

三种映射的对比：

维度	直接映射	组相联	全相联
位置数	1	N	全部
比较器数	1	N	全部
冲突 miss	**最多**	中	**最少**
硬件成本	最低	中	最高
典型用途	L1（简单）	L1/L2（主流）	TLB、小 cache

15.4 替换与写策略

策略	做法	优点	缺点
LRU	换最久没用的	最贴合时间局部性	实现贵（要记顺序）
FIFO	换最早进来的	简单	可能换掉常用的
随机	随便换	最便宜	不可预测

策略	做法	优点	缺点
Write-through	写时同时写内存	简单、内存始终最新	写流量大
Write-back	只写 cache，置 **dirty bit**，驱逐时才写回	写流量小	需要 dirty bit，一致性复杂

> notes 的判断：实践中 cache **几乎总是**用 LRU（或 FIFO）+ write-back，其他组合“not considered reasonable”。

15.5 三类（四类）miss

类型	别名	原因	能否避免
Compulsory	cold miss	第一次访问	不能（除非预取）
Capacity	—	cache 太小，块被挤出去	只能靠增大容量
Conflict	collision	多个块映射到同一组	提高相联度
Coherence	communication	别的核写了共享数据	见第 17 章

15.6 优化手段与代价（notes 表 10.3 的结论）

手段	Hit time	Miss penalty	Miss rate
增大 block size		−	+（到拐点后转差）
增大 cache 容量	−		+
提高相联度	−		+
多级 cache		+	
索引时避开地址翻译（VIPT）	+		

（`+` = 改善，`−` = 变差）

Block size 的非单调性是重点：一开始增大 block 利用空间局部性 → miss rate 下降；但块太大导致 cache 里块数变少 → 冲突 miss 上升，而且每次 miss 要搬更多数据 → **AMAT 反而上升**。

15.7 Cache blocking（分块）

矩阵乘法 `C = A×B` 的三重循环，直接写会反复换入换出。**分块**把矩阵切成能塞进 cache 的小块，让一个块在被驱逐前被尽量多用几次：

notes 的关键提醒：**blocking 是程序员的责任，gcc 不会替你做**——因为编译器不知道你的数据布局意图。

15.8 本章陷阱

1. AMAT 写成 `MissRate × MissPenalty`（漏了 hit time）。
2. 两级 cache 的 L2 miss rate 当成全局比例。
3. index 位数用“字节数”而非“块数”算。
4. tag 位数忘记减 offset 与 index。
5. 认为直接映射可以选替换策略（不能）。
6. 认为 write-back 不需要写内存（驱逐时要写）。
7. block size 越大越好（有拐点）。
8. 忘记 valid bit / dirty bit。

第 16 章 虚拟内存与 TLB

> 来源：notes §12.1（vm、address-translation、page-table、memory-manager）、§12.2（tlb、memory-hierarchy-full）；对应 Lecture 36—37、Homework 10、Discussion 12—13。

16.1 动机：三个问题一起解决

问题	虚拟内存的答案
程序互相踩内存	每个进程一套**独立**虚拟地址空间
内存不够用	不常用的页放到磁盘（demand paging）
程序里的地址在编译时未知	链接时用虚拟地址，运行时再翻译

16.2 地址翻译

```
虚拟地址 VA:
+---------------------+-------------+
|   VPN (virtual pn)  | page offset |
+---------------------+-------------+
          |                   |
          v                   |
   +--------------+           |
   |  Page Table  |           |
   +--------------+           |
          | PPN               |
          v                   v
+---------------------+-------------+
|   PPN (physical pn) | page offset |  = PA
+---------------------+-------------+
```

四步（notes §12.1）：

1. 从 VA 取出 **VPN**（高位）；
2. 用 VPN 查**页表项 PTE**；
3. 若 PTE **valid** → 取出 **PPN**，与 offset 拼成 **PA**；若 **invalid** → **page fault**，从磁盘调入后重试；
4. 用 PA 访问内存。

页大小与位数：4 KiB 页 = `2^12` B → offset **12 位**。32 位 VA → VPN **20 位**。

例（notes 原文）：`0xFFFFF004` → VPN=`0xFFFFF`、offset=`0x004`；PTE 给出 PPN=`0x1` → PA=`0x1004`。

page fault 的代价：notes 写作“about a million cycles later”。

16.3 页表设计

字段	作用
PPN	物理页号
valid	页是否在内存
dirty	是否被写过（换出时要不要写回）
access / permission	读/写/执行权限

层级页表（hierarchical page table）在本课 out of scope（notes 明确写出），只需理解单级页表 + TLB。

16.4 TLB：地址翻译的 cache

核心痛点：页表在**内存**里（1 MiB 的页表塞不进 128 KiB 的 L1），所以每次访存要**访问内存两次**（查页表 + 取数据），几百周期。

TLB（Translation Lookaside Buffer）缓存 VPN → PPN 的映射。

```
VA --> [ TLB 查 VPN ] --hit--> PPN --> PA --> [ cache/内存 ]
            |
          miss
            v
      [ 页表 walk ] --> 更新 TLB
```

参数	课程的典型值
条目数	32–128
相联度	**全相联**（本课假设）
替换	FIFO 或随机
每核数量	1

TLB reach（覆盖范围）：

```
TLB reach = (# TLB entries) × (page size)
```

例：64 项 × 4 KiB = 256 KiB。若工作集超过它，TLB 就会 thrash。

上下文切换怎么办：

方案	做法	代价
全部 flush	换进程时清空 TLB	简单，但换回后要重新预热
加 PID/ASID	每个条目记进程号，比较时同时比 PID	硬件复杂，但不用 flush

16.5 三种情形的代价（notes 表 12.1）

情形	TLB	页表（内存）	磁盘	代价
1 TLB 命中	✅	不访问	不访问	**~1 周期**
2 TLB miss，页在内存	❌	✅ valid	不访问	**~100 周期**
3 page fault	❌	❌ invalid	✅ 访问	**~1000+ 周期**

一条重要推论（notes 的 Quick Check）：**TLB 命中 ⇒ 数据一定在内存**。因为 TLB 里的条目来自 valid 的 PTE。

16.6 完整层次：一次 load 要过几关

```
VA
 |
 +--> TLB --miss--> 页表(内存) --invalid--> 磁盘 --> 更新 PTE/TLB
 |      |
 |     hit
 v      v
PA --> L1 --> L2 --> L3 --> DRAM
```

优化	作用
VIPT（索引用虚拟地址、tag 用物理地址）	让 cache 索引与 TLB 翻译**并行**，省掉 hit time
多级页表	省页表空间（本课不深入）
大页（huge page）	提高 TLB reach

16.7 本章陷阱

1. offset 位数算错（4 KiB 页 = 12 位）。
2. 把 VPN 当 VA（VPN 是高位部分）。
3. 认为 TLB miss 一定是 page fault（多数只是要 walk）。
4. TLB reach 用字节数当条目数。
5. 忘记 page fault 后要**重新执行**那条指令。
6. 认为 TLB 是数据 cache（它缓存的是**翻译**）。

第 17 章 并行：Amdahl、ILP、SIMD 与 TLP

> 来源：notes §11.1 Amdahl's Law（parallel-performance）、§11.2 Flynn/SIMD（parallel-dlp）、§11.3 MIMD/OpenMP/coherence（parallel-tlp）；对应 Lecture 30—35、Homework 9、Lab 7。

17.1 动机：为什么单核不再变快

notes 的叙事主线：1990s—2000s 靠**加深流水线 + 提频**提升性能，但功耗密度（power density）随频率飙升，最终撞上**功耗墙（power wall）**；同时 Moore's Law 也放缓。于是业界转向**并行**。

17.2 Amdahl 定律

```
Exec_time_with_improvement = (受影响部分 / 加速倍数) + 未受影响部分

Speedup = 1 / [ (1 − f) + f / s ]        f = 可优化比例, s = 该部分的加速倍数
```

并行特例（P 个处理器、串行比例 s_seq）：

```
Speedup(P) = 1 / [ s_seq + (1 − s_seq)/P ]  →  1 / s_seq   (P → ∞)
```

必背结论：加速比的上限由串行部分决定。P 无穷大时最多快 `1/(串行比例)` 倍。

例题（notes Quick Check）：128 核、想拿 10× 加速 →

```
10 = 1 / [x + (1−x)/128]  =>  x ≈ 9%
```

即**串行部分只能占 9%**。这就是 Amdahl 被称为“heartbreaking law”的原因。

17.3 三种并行

17.3.1 ILP（指令级并行）

同一线程内多条指令重叠执行。手段：**流水线**、**超标量（superscalar）**、乱序执行。

```
超标量：CPI < 1（每周期发射 > 1 条）
```

17.3.2 DLP / SIMD（数据级并行）

一条指令作用在**一个向量**上：

```
标量:  c[i] = a[i] + b[i]      一条指令一个元素
SIMD:  C    = A    + B         一条指令 N 个元素
```

Intel 的演进（notes）：**MMX（64 位，1997）→ SSE（128 位，1999–2000）→ AVX（256 位，2011）→ AVX-512（512 位）**。

SIMD 的三重收益（notes 原文归纳）：

1. **只取指/译码一次**（摊薄前端开销）；
2. 各数据流**互相独立**，天然并行；
3. 利用**空间局部性**，一次搬一整块。

C 里通过 **intrinsics** 使用：

```
#include <immintrin.h>
__m256 a = _mm256_loadu_ps(A + i);
__m256 b = _mm256_loadu_ps(B + i);
__m256 c = _mm256_add_ps(a, b);
_mm256_storeu_ps(C + i, c);
```

> **注意**：SIMD 与 vector architecture 不同（notes 明确说区别超出本课范围）：真正的向量机支持 `reduce-add` 之类的“向量→标量”操作，纯 SIMD 不支持。

17.3.3 TLP（线程级并行）

多核 + 多线程。三种模型：

模型	例子	特点
共享内存 + 线程	pthreads、OpenMP	编程简单，要处理竞争
消息传递	MPI	无共享状态，扩展性好
数据并行框架	MapReduce、CUDA	表达受限但易扩展

OpenMP 最小例子：

```
#pragma omp parallel for
for (int i = 0; i < n; i++)
    c[i] = a[i] + b[i];
```

17.4 共享内存的两个硬问题

17.4.1 数据竞争（data race）

```
// 错误：多线程同时 counter++
counter++;      // 读-改-写，三步不是原子的
```

解法：**临界区（critical section） + 锁或原子指令（AMO，atomic memory operation）**。

17.4.2 Cache 一致性（cache coherence）

三核访问同一地址 `0x5000`：

```
CPU1 读 -> 拿到 20，缓存
CPU2 读 -> 拿到 20，缓存
CPU1 写 40 -> CPU1 的 cache 更新，CPU2 的 cache 变"脏"
```

定义（notes 引 Wikipedia）：**coherent 系统里，同一内存区域的多个缓存副本必须相同**。

第四类 miss：**coherence miss**（communication miss）——因为别的核写了共享数据，导致本地副本失效而 miss。

写失效协议（write-invalidate）：写者让其他副本失效；读者下次访问就 miss 并从写者/内存取新值。notes 把 snooping 协议的细节列为 **out of scope**，只需理解“失效 + 重新取”的思想。

17.4.3 伪共享（false sharing）

两个线程改**同一个 cache line 里的不同变量** → 缓存行在两核之间来回弹，性能崩塌。解法：填充（padding）到不同 cache line。

17.5 本章陷阱

1. Amdahl 的 `f` 与 `s` 搞反。
2. 认为核数翻倍就能加速翻倍。
3. SIMD 宽度与数据类型不匹配（`_mm256` 装 8 个 float，不是 8 个 double）。
4. 忘记对齐要求（`loadu` vs `load`）。
5. 数据竞争只在“看起来无害”的 `++` 上出现。
6. 把 coherence 与 consistency 混为一谈（前者管“副本是否相同”，后者管“读写顺序”）。

第 18 章 三个项目精讲

> 来源：`https://cs61c.org/fa26/projects/proj1/`、`/proj2/`（含 `part-a`、`part-b`、`calling-convention`、`function-definitions`）、`/proj3/`（含 `part-a`、`part-b`、`testing`）；全部于 2026-11 访问。

18.1 依赖关系

```
Lab 0/1/2 (C, gdb, valgrind) ──> Project 1 snek (C, 堆, 文件)
Lecture 9-14 + Lab 3/4       ──> Project 2 CS61Classify (汇编, 调用约定)
Lecture 16-22 + Lab 5        ──> Project 3A CS61CPU (ALU/RegFile/addi)
Lecture 23-25                ──> Project 3B (全指令 + 2 级流水)
```

18.2 Project 1：`snek`

目标：用 C 实现一个可玩的贪吃蛇，练习**堆内存管理、二维字符数组、文件读入、多蛇移动**。

Task	函数	考点
1	`create_default_game`	`malloc` + 硬编码 18×20 盘面
2	`free_game`	逐层释放（每行 → 行数组 → snakes → game）
3	`print_board`	`fprintf` 到 `FILE*`
4	`update_game`（含 4.1–4.5）	字符分类、`next_square`、`update_head`、`update_tail`
5	`load_board`（`read_line`）	**只能用 `fgets`**；行长不一；`realloc`
6	`initialize_snakes`（`find_head`）	从尾部沿方向字符走到头
7	`main`（`snake.c`）	单步推进
8	反馈表	不计分

数据结构（`game.h`，不可修改）：

```
typedef struct {
    unsigned int tail_row, tail_col;
    unsigned int head_row, head_col;
    bool live;
} snake_t;

typedef struct {
    unsigned int num_rows;
    char **board;              // 每行以 '\n' 结尾且是合法字符串
    unsigned int num_snakes;
    snake_t *snakes;
} game_t;
```

必须遵守的硬性约束：

- **必须在 hive 机器上完成**，且必须提交 Provenance log（缺失 −100 分）；
- **只能改** `game.c`、`snake.c`、`custom_tests.c`；
- `load_board` **必须用** `fgets`，不能 seek/rewind（要支持 stdin）；
- 内存要“按需分配”：行长 3 就不能分配 100 字节；
- 用 `make valgrind-test-free-game` 查泄漏。

测试：`make unit-tests`、`make run-integration-tests`、`make run-nonexistent-input-file-test`；共 21 个集成测试（`01-simple` … `21-bigL`）。

18.3 Project 2：`CS61Classify`

目标：用 RISC-V 汇编写一个 MNIST 手写数字分类器（推理，不训练）。

Part A（函数库）

Task	函数	参数	错误码
1	`abs`	`a0` = 指针，结果写回	—
2	`relu`	`a0` =数组，`a1` =长度	**36**（长度 < 1）
3	`argmax`	`a0` =数组，`a1` =长度 → `a0` =下标	36
4	`dot`	`a0,a1` =两数组，`a2` =元素数，`a3,a4` =stride	36（n<1）、**37**（stride<1）
5	写测试	覆盖率必须 **100%**	26（malloc 失败）

Part B（矩阵与分类）

Task	函数	要点
6	`matmul`	行主序；`a0..a6` 七个参数；维度不匹配 → **38**
7	`read_matrix`	从文件读矩阵
8	`write_matrix`	写回文件
9	`classify`	串起 `matmul` → `relu` → `matmul` → `argmax`
10	反馈表	—

关键工程纪律（OH Policy 与调用约定页）：

- 每个寄存器都要写**注释**说明用途（“this holds i”太笼统，要具体）；
- 必须用 **calling convention checker** 并清零所有报错；
- 必须用 **memcheck** 并清零所有内存错误；
- 参考 `bash test.sh` 的子命令：`part_a`、`test_<func>`、`coverage`；memcheck 加 `-mc` / `-mcv`。

Venus 工作流：

```
bash test.sh download_tools          # 只跑一次
java -jar tools/venus.jar . -dm      # 挂载当前目录
# 浏览器打开 https://venus.cs61c.org，终端里 mount local vmfs
vdb test_relu_standard.s             # 在 test-src 里启动调试器
```

18.4 Project 3：`CS61CPU`

目标：在 **Logisim Evolution** 里搭一个能跑真实 RV32I 指令的 CPU。

Part A（20 分 = 项目 20%）

Task	电路	测试	分值
1	`alu.circ`	`alu-all` / `alu-logic` / `alu-mult` / `alu-shift` / `alu-slt-sub-bsel`	7
2	`regfile.circ`	`regfile-more-regs` / `regfile-read-only` / `regfile-read-write` / **`regfile-x0`**	8
3	`imm-gen.circ`	I 型（Part A 只需 `addi`）	—
4	`cpu.circ`	`test_addi`：`addi-basic` / `addi-negative` / `addi-positive`	5

Part B（80 分 = 项目 80%）

Task	内容
5	I 型全指令（`andi` / `ori` / `xori` / `slli` / `srli` / `srai` / `slti`）+ 控制逻辑
6	R 型（`add` … `slt`）+ `mul` / `mulh` / `mulhu`
7	B 型（6 条）+ **branch comparator**
8	Load/Store（`lb/lh/lw/sb/sh/sw`）+ **partial load/store**
9	跳转与 U 型（`jal` / `jalr` / `auipc` / `lui`）
10	**2 级流水线** + 控制冒险 flush
11	反馈表（2 分）

评分构成（Part B）：指令 **1.5 分/条 × 36 条 = 54**；集成测试 **2 分/个 × 12 = 24**；反馈表 **2**；合计 **80**。

控制逻辑实现路线（Task 5.2 的建议）：

```
1. 在电子表格里填真值表（每行一条指令 × 9 个控制信号）
2. 把 "ROM Output" 列拷进 Logisim 的 ROM
3. 用 Priority Encoder 选行（输入是"是不是这条指令"的独热信号）
4. PCSel 用组合逻辑单独做（因为它依赖 BrEq/BrLT）
```

流水线的关键提示（Task 10）：

- 只有**数据从左到右**的路径需要插流水寄存器；**反馈路径**（WB 写 RegFile、MEM 定 PC）**不插**；
- 阶段间不同的值很少（主要是 **PC**）；

- 分支/跳转要 **flush 阶段 1**：把取到的指令换成 `nop = addi x0,x0,0 = 0x00000013`；
- 用 `-p` / `--pipelined` 跑流水版测试；**流水版不再通过非流水版测试**。

第 19 章 与已有笔记的衔接，以及复习与自测

19.1 与同目录其他笔记的分工

主题	本笔记（CS61C）	CSAPP（15–213）	15–418	CS162
数据表示	补码/浮点**编码手算**	补码、溢出、浮点（第 2 章）更偏 C 语义	—	—
汇编	**RISC-V**，手写 + 调用约定	**x86-64**，读编译器输出	—	少量
链接	CALL 四步、重定位表（第 10 章）	**第 7 章**最详（ELF、符号解析、重定位、动态链接）	—	—
存储层次	cache 映射/AMAT/TLB（第 15–16 章）	**第 6 章**（cache 与性能优化）	cache 相干性、伪共享的**工程后果**	—
虚拟内存	VA→PA、TLB、page fault	**第 9 章**（页表、地址翻译、malloc）	—	页表/置换策略/请求调页（**内核视角**）
并行	Amdahl、SIMD、TLP、coherence 概念	第 12 章（并发编程）	**主力**：ILP、SIMD、GPU、work stealing	线程/锁/调度
CPU 设计	**单周期 + 五级流水**（本课独有）	第 4 章（Y86 流水线，相近）	—	—
数字逻辑	**门/FSM/ALU**（本课独有）	附录 A（较简）	—	—

建议的阅读顺序：

1. 先用本笔记第 2–9 章建立“比特 → 汇编”的直觉；
2. 用 CSAPP 第 3、6、7 章补齐 **x86 视角与链接细节**（CS61C 的 CALL 只讲到概念层）；
3. 用本笔记第 13–16 章建立“数据通路 → 流水线 → cache”的硬件直觉；
4. 用 15–418 笔记理解 **cache 一致性与并行性能**的真实工程后果；
5. 用 CS162 笔记把 **TLB 与页表**从“翻译机制”上升到“内核数据结构”。

三处最容易被混淆的概念：

容易混淆	正确区分
cache 的“block”vs 虚拟内存的“page”	block 由硬件管理、几十字节；page 由 OS 管理、几 KiB
TLB miss vs page fault	TLB miss 多数只需 walk 页表；page fault 要访问磁盘
coherence vs consistency	coherence 管“同一地址的多个副本是否相同”；consistency 管“读写操作的可见顺序”

19.2 复习清单（按讲次打勾）

数字与 C（讲 2–8）

- [ ] 用 `ceil(log2 k)` 算位宽；解释 4 位无符号溢出
- [ ] 手画补码 number wheel，标出溢出位置
- [ ] 由公式 `-2^{n-1}d_{n-1} + Σ` 推出“取反加一”
- [ ] 手算 `-12.375` 的单精度编码与反解
- [ ] 说出 denorm 的隐含指数（−126）与最小 denorm（2^−149）
- [ ] 说出四种舍入模式与默认模式
- [ ] 画出 C 内存四段布局
- [ ] 解释 `a[i]` 与 `*(a+i)` 的等价性
- [ ] 写出函数指针的声明与调用

RISC-V（讲 9–14）

- [ ] 默写 32 个寄存器的 ABI 名与 saver
- [ ] 写出六步调用中 caller/callee 各做什么
- [ ] 画出栈帧并写出序言/尾声
- [ ] 手写递归 `factorial` 并解释为什么必须存 `ra` 与 `n`
- [ ] 画出六种指令格式的位域
- [ ] 重组 B 型与 J 型立即数
- [ ] 解释 `li` 的 `hi20` 进位修正
- [ ] 区分 `slt/sltu`、`sra/srl`、`lb/lbu`

CALL（讲 15）

- [ ] 说出四步的输入输出
- [ ] 列出 `.o` 的六个组成部分
- [ ] 解释为什么汇编器要两遍扫描
- [ ] 说出哪些指令需要 linker 重定位（B 型不需要）

- [ ] 对比静态/动态链接

数字逻辑与数据通路（讲 16–22）

- [ ] 由真值表写 SOP 并用布尔代数化简
- [ ] 用 `Sub` 一位控制实现加/减一体
- [ ] 画出 FSM 的“状态寄存器 + CL”结构
- [ ] 由 `T >= t_cq + t_cl + t_setup` 解释时钟周期
- [ ] 画出单周期数据通路并标出 9 个控制信号
- [ ] 填出 `add/addi/lw/sw/beq/jal/lui` 的控制信号行
- [ ] 解释 `PCSel` 为什么不能进 ROM

流水线与 cache（讲 23–29）

- [ ] 说出五级与四个流水寄存器各存什么
- [ ] 画 waterfall 图判断 RAW 冒险
- [ ] 解释 forwarding 的两条路径与 load-use 的必然停顿
- [ ] 计算含 stall 的 CPI
- [ ] 由容量/块大小/相联度算出 tag/index/offset 位数
- [ ] 追踪一串访存的 hit/miss 并维护 dirty bit
- [ ] 用 AMAT 比较单级与两级 cache
- [ ] 说出 block size 增大为何 miss rate 非单调

VM 与并行（讲 30–37）

- [ ] 画 VA→PA 的翻译图
- [ ] 算 TLB reach
- [ ] 说出 TLB 命中/未命中/page fault 的三种代价
- [ ] 用 Amdahl 算加速比与上限
- [ ] 区分 ILP/DLP/TLP
- [ ] 解释 coherence miss 与写失效协议

19.3 自测题（12 题，附答案）

Q1. 8 位补码 `0xF6` 的十进制值是多少?

> ▸ **答案**
>
> `0xF6 = 1111 0110`。最高位为 1 → 负数。取反加一：`0000 1001 + 1 = 0000 1010 = 10` → **−10**。

Q2. 单精度浮点 `0x41C80000` 的值是多少?

> **答案**
>
> `0 10000011 1001000...`：s=0，exp=131→`131−127=4`，尾数 $1.1001_2 = 1.5625$；$1.5625 \times 16 = 25.0$。

Q3. 为什么 `lw` 之后紧跟使用该寄存器的指令必须停 1 拍，而 `add` 之后不用？

> **答案**
>
> `add` 的结果在 **EX 末**就有，可通过 EX/MEM→EX 前递直接给下一条的 EX。`lw` 的数据要到 **MEM 末**才拿到，而下一条的 EX 与它的 MEM 同周期，前递来不及 → 必须停 1 拍（load-use hazard）。

Q4. 一个 32 KiB、4 路组相联、64 B block 的 cache，地址 32 位。求 index 与 tag 位数。

> **答案**
>
> 块数 = 32768/64 = 512；组数 = 512/4 = **128** → index = **7 位**；offset = log2(64) = **6 位**；tag = 32 − 7 − 6 = **19 位**。

Q5. 上题 cache，访问地址 `0x0000_1F40`，求 index 与 offset。

> **答案**
>
> offset = `0x40 & 0x3F` = **0**（低 6 位 `000000`）；index = `(0x1F40 >> 6) & 0x7F` = `0x7D & 0x7F` = **0x7D = 125**；tag = `0x1F40 >> 13` = **0**。

Q6. L1: hit 2 周期, miss 8%, penalty 100；加 L2: hit 10 周期, miss 20%, penalty 100。求两种 AMAT。

> **答案**
>
> 单级：`2 + 0.08×100 =` **10 周期**。 两级：`2 + 0.08×(10 + 0.20×100) = 2 + 0.08×30 =` **4.4 周期**。

Q7. 95% 可并行的程序，用 64 核最多快多少？上界是多少？

> **答案**
>
> `1/(0.05 + 0.95/64) = 1/(0.05+0.01484) ≈` **15.4×**；`P→∞` 上界 `1/0.05 =` **20×**。

Q8. `li x5, 0xDEADBEEF` 展开成哪两条指令？

> ▸ **答案**
>
> `lo12 = 0xEEF`，其最高位为 1 → `hi20 = (0xDEADB + 1) = 0xDEADC`。
>
> ```
> lui  x5, 0xDEADC
> addi x5, x5, 0xEEF
> ```
>
> 验证：`0xDEADC000 + 0xFFFFFEEF = 0xDEADBEEF` ✅

Q9. 为什么 `beq` 的 `ASel` 要选 PC 而不是 `rs1`？

> ▸ **答案**
>
> 因为分支目标是 `PC + imm`（PC 相对），ALU 要做的是“PC 加偏移”，而不是“rs1 加偏移”。`rs1` / `rs2` 只用于 Branch Comparator 比较。

Q10. TLB 命中是否意味着数据在 cache 里？

> ▸ **答案**
>
> **不意味着**。TLB 命中只保证“翻译有效、页在内存”，数据仍可能在 cache 里 miss，需要去内存取。

Q11. 32 位 VA、4 KiB 页、TLB 64 项全相联。TLB reach 是多少？

> ▸ **答案**
>
> `64 × 4 KiB = 256 KiB`。即最多能同时“免 walk”覆盖 256 KiB 的虚拟地址空间。

Q12. 为什么流水线 flush 时要把 `RegWEn` 与 `MemRW` 一起清零？

> ▸ **答案**
>
> 因为被 flush 的指令已经进入流水线，若其控制信号仍为 1，会在后续周期**真的写寄存器或写内存**，造成不可恢复的状态污染。`nop`（`addi x0,x0,0`）天然 `RegWEn=1` 但写的是 `x0`（被丢弃）；更稳妥的做法是直接把控制信号清零。

第 20 章 参考来源

> 全部 URL 于 **2026-11** 访问。凡标注“正文不可提取”者，表示该页仅含索引/链接而无正文文本。

20.1 课程主页与政策

内容	URL	状态
课程主页（Fall 2026）	https://cs61c.org/fa26/	✅ 200，含 40 讲日程表、lab/hw/project 日期
课程总入口	https://cs61c.org/	✅ 200
Policies（先修/评分/考试/LLM 政策/教材）	https://cs61c.org/fa26/policies/	✅ 200，本文评分与考试信息的唯一来源
历年真题索引	https://cs61c.org/fa26/resources/exams/	✅ 200，含 2018–2026 各学期链接与 Q&A
资源页（Reference Card v4.0.1、GDB card、Venus reference）	https://cs61c.org/fa26/resources/	✅ 200
常见错误	https://cs61c.org/fa26/resources/common-errors/	✅ 200
课程归档	https://inst.eecs.berkeley.edu/~cs61c/archives.html	✅ 200

20.2 讲义页（40 个，正文不可提取）

内容	URL	状态
`lec01` – `lec40`	`https://cs61c.org/fa26/lectures/lecNN/`（NN = 01…40）	✅ 全部 200，但**每页约 6 KB，仅含讲题 + Slides/Drive/Readings 链接，无可提取正文**

> 因此本文的**技术内容**全部以 notes.cs61c.org 的章节正文为依据；讲义页只用于确认“讲题与讲次的对应关系”。

20.3 课程笔记（notes.cs61c.org，本文的技术依据）

章节	URL
站点首页 / 目录	https://notes.cs61c.org/

章节	URL
sitemap（156 个页面全量）	https://notes.cs61c.org/sitemap.xml
§1 数字表示	https://notes.cs61c.org/content/number-rep/ | `/binary-decimal-hex` | `/integer-representations` | `/summary`
§2 C 语言	`/content/c-basics/`（含 `/compile-vs-interpret`、`/c-vs-java`、`/c-variables`、`/c-syntax`）| `/content/c-pointers-arrays-strings/`（`/pointers`、`/arrays`、`/strings`）| `/content/c-memory-management/`（`/stack`、`/heap`、`/heap-allocator`）| `/content/c-generics/`（`/generics`、`/fn-ptr`）| `/content/c-odds-and-ends/`（`/endianness`、`/ll-add-front`、`/ll-full`）
§3 浮点	`/content/floating-point/`（`/fp-floating-point`、`/fp-examples`、`/fp-special-numbers`、`/fp-discussion`、`/summary`）
§4 RISC-V	`/content/rv-intro/`（`/elements`、`/rv-isa`、`/rv-arithmetic`、`/rv-bitwise`）| `/content/rv-data-transfer/`（`/rv-load-store`、`/examples`、`/example-sp`）| `/content/rv-decision-making/`（`/branches`、`/loops`）| `/content/rv-procs/`（`/rv-jumps-stack`、`/rv-calling-conv`、`/examples`）
§5 指令格式	`/content/rv-isa-formats-i/`（`/r-type`、`/i-type`、`/s-type`）| `/content/rv-isa-formats-ii/`（`/b-type`、`/j-type`、`/u-type`）| `/content/misc/rv32i-green-card`
§6 CALL	`/content/call/`（`/assembler-object-file`、`/linker-resolve-references`、`/example`、`/interpret-translate`）
§7 数字逻辑	`/content/sds-intro/`（`/transistors`）| `/content/sds-combinational-logic/`（`/boolean-algebra`、`/cl-design`）| `/content/sds-state/`（`/register`、`/timing-synchronous-system`、`/fsm`、`/pipelining`）| `/content/sds-blocks/`（`/adder-subtractor`、`/alu`）
§8 数据通路与控制	`/content/datapath/`（`/elements`、`/r-type`、`/i-type`、`/load-store`、`/b-type`、`/jump`、`/u-type`、`/control`、`/instruction-timing`、`/special-regs`）
§9 流水线与冒险	`/content/pipeline/`（`/laundry`、`/five-stage-pipeline`、`/iron-law-cpi`、`/superscalar`、`/energy`）| `/content/pipeline-hazards/`（`/structural-hazards`、`/data-hazards`、`/control-hazards`）
§10 Cache	`/content/caches-intro/`（`/memory-hierarchy`、`/amat`）| `/content/caches-ii/`（`/fully-associative`、`/direct-mapped`、`/set-associative`）| `/content/caches-iii/`（`/optimizations`、`/blocking`）
§11 并行	`/content/parallel-performance/`（`/optimization-gcc`、`/dgemm`、`/dgemm-sisd`）| `/content/parallel-dlp/`（`/simd`、`/dgemm-simd`、`/intrinsics`）| `/content/parallel-tlp/`（`/threads`、`/openmp`、`/openmp-examples`、`/dgemm-mimd`、`/data-race`、`/cache-coherence`、`/locks-amos`）

章节	URL
§12 虚拟内存	`/content/vm/`（`/address-translation`、`/memory-manager`、`/page-table`、`/tlb`、`/memory-hierarchy-full`）
§13 前缀与参考	`/content/misc/iec-prefixes` | `/content/reference`

> 说明：上表中以 `/content/...` 开头的是相对 `https://notes.cs61c.org` 的路径。**注意**：`/content/rv-isa-formats-ii/b-type` 与 `/j-type` 两页正文开头明确标注这些笔记尚未完成（页面原文为“These notes are incomplete; see Spring 2026 lecture”），即 **B/J 型格式的笔记本身未写完**，但页面同时声明“lecture is still in scope”。本文的 B/J 立即数重组表依据 green card 的格式总表与 S 型的对比推得，属于对官方格式表的直接转写。

20.4 项目页

项目	URL	状态
Project 1 `snek`	https://cs61c.org/fa26/projects/proj1/	✅ 200（约 61 KB，含 Setup/Provenance/8 个 Task/Submission）
Project 2 `CS61Classify`	https://cs61c.org/fa26/projects/proj2/ | `/part-a/` | `/part-b/` | `/calling-convention/` | `/function-definitions/`	✅ 全部 200
Project 3 `CS61CPU`	https://cs61c.org/fa26/projects/proj3/ | `/part-a/` | `/part-b/` | `/testing/`	✅ 全部 200
Project 4 `61kaChow`（可选）	主页标注为 OPTIONAL；`/fa26/projects/proj4/` 返回 404	⚠️ 无独立页面

> 注：`/fa26/projects/project1/`、`/project2/`、`/project3/` 三种旧式路径均返回 **404**；有效路径是 `proj1`、`proj2`、`proj3`。

20.5 工具与环境

内容	URL	状态
Venus 在线模拟器	https://venus.cs61c.org/	✅ 200

内容	URL	状态
Logisim Evolution（课程分发版）	由各项目仓库 `bash test.sh download_tools` 获取（`tools/logisim-evolution.jar`）	需运行脚本
Gradar（组队登记）	https://gradar.cs61c.org/	✅ 200
课程 Ed	https://edstem.org/us/courses/102678/	需登录
OH Queue	https://oh.cs61c.org/	✅ 200

20.6 教材（Policies 页 *Reading* 一节）

缩写	书目
P&H	Patterson & Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed., ISBN 0128203315
K&R	Kernighan & Ritchie, *The C Programming Language*, 2nd ed.
WSC	Barroso, Clidaras, Hölzle, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*, Morgan & Claypool, 2013

20.7 未能获取或存在不确定性的材料（诚实清单）

项	情况
40 讲讲义正文 / PDF	讲义页只有 Google Slides / Drive / Askademia 链接，**无法以文本形式抓取**；本文技术内容改以 notes 章节为依据
Project 4 `61kaChow` 的 spec	主页只写“OPTIONAL: Lab 8 / Project 4: 61kaChow”，`proj4` 路径 404，**无 spec 可读**
Lecture 35/38/39（guest lectures）的具体内容	仅知讲题（Parallelism VI；Kathy Yelick；Dave Patterson），无 notes 对应章节
B 型/J 型格式笔记	notes 页面自述“尚未完成（incomplete）”，本文按 green card 格式表整理
2026 秋季之前的真题 PDF	索引页可访问，但历年卷面内容需逐个点击，本文未逐份解析
Workbook/讨论课答案	部分 worksheet 需登录 bCourses，未获取

文档结束。若课程当学期（Fall 2026）后续调整讲次、截止日期、评分权重或项目 spec，**一律以 https://cs61c.org/fa26/ 与 Policies 页为准**。